

사물 탐지 모델(R-CNN,SSD,YOLO)

Object Detection Model
(R-CNN,SSD,YOLO)

사물 탐지 모델

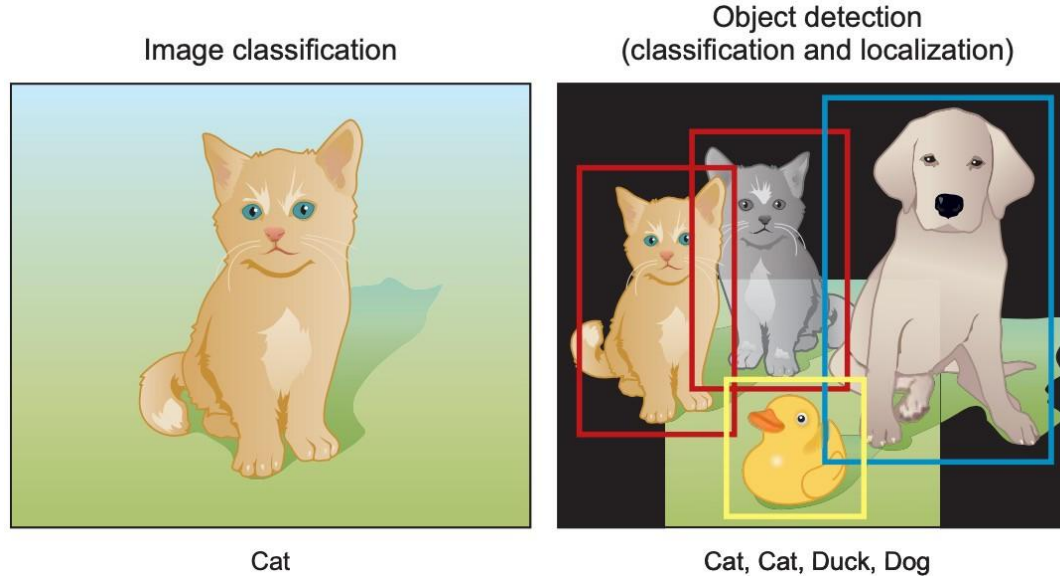
학습 목표

- 이미지 분류와 사물 탐지의 차이를 이해한다.
- 사물 탐지 모델의 알고리즘과 평가 방법을 이해한다.
- R-CNN, SSD, YOLO 등의 사물 탐지 알고리즘을 적절히 사용한다.

사물 탐지 모델

학습 내용

- **이미지 분류** Image Classification 에서는 이미지에 주 대상 물체가 하나만 있다고 가정하며, 모델의 역할은 이 대상 물체의 카테고리가 무엇인지 식별하는 것
- **사물 탐지** Object Detection 에서는 이미지에 하나 이상의 대상이 존재하고, 이미지상에서 대상의 카테고리 뿐만 아니라 위치도 함께 특정해야 하는 경우



사물 탐지 모델

이미지 분류와 사물 탐지의 차이점

이미지 분류	사물 탐지
이미지에 있는 (하나뿐인) 대상의 클래스를 예측하는 것이 목표 • 입력: 하나의 대상이 있는 이미지 • 출력: 클래스 레이블 (고양이, 개, 등) • 출력 예: 각 클래스에 속할 확률 (고양이 84%)	이미지에 있는 (하나 이상의) 대상의 위치를 나타내는 박스와 해당 물체의 클래스를 예측하는 것이 목표 • 입력: 하나 이상의 대상이 있는 이미지 • 출력: 각 대상의 위치를 특정하는 경계박스 (좌표)와 해당 물체의 클래스 레이블 • 출력 예: - box1의 좌표(x,y,w,h)와 클래스에 속할 확률 - box2의 좌표와 클래스에 속할 확률 좌표(x,y,w,h)에서 x와 y는 경계 박스의 중심점 좌표고, w와 h는 경계 박스의 너비와 높이이다.

사물 탐지의 응용

- 자율주행차 기술: 주변 영상으로부터 가까이에 위치한 차량, 보행자, 도로 및 장애물의 위치 인식
- 보안 기술: 침입자나 폭탄 등 비정상적인 대상 인식

1. 사물 탐지 알고리즘

사물 탐지 알고리즘의 일반적인 프레임워크

사물 탐지 프레임워크는 다음 4개 요소로 구성됨

① 영역 제안 ② 특징 추출 및 예측 ③ 비최대 억제 ④ 평가지표

각 구성 요소의 간략 설명

- **영역 제안** Region Proposal : 이미지에서 시스템이 처리할 영역인 **RoI** Regions of Interest
를 제안하는 딥러닝 모델 또는 알고리즘. RoI는 이미지 내 물체가 존재할 것이라 예상되는 영역을 의미함.
이 컴포넌트는 각각 물체 존재 **확신도** objectness score 가 매겨진 많은 수의 박스 정보를 출력함. 이 중 높은 물체 존재 확신도가 매겨진 박스는 추가 처리를 위해 전달됨.
- **특징 추출 및 예측** : 각 박스 영역의 시각적 특징이 추출됨.
이러한 특징을 평가해서 물체 존재 여부와 물체의 클래스를 판단함

1. 사물 탐지 알고리즘

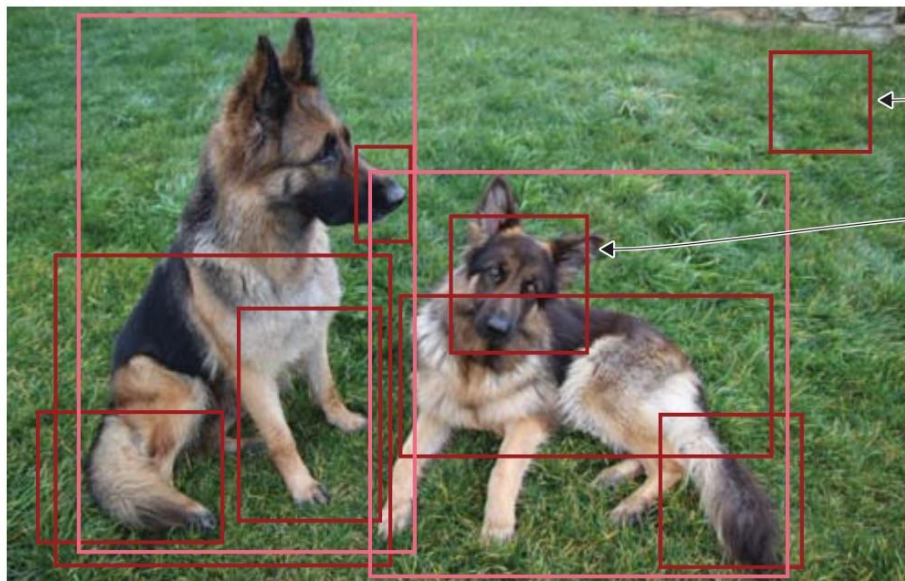
사물 탐지 알고리즘

- 각 구성 요소의 간략 설명
 - **비최대 억제** non-maximum suppression, NMS : 이 단계쯤 이면 모델이 같은 물체에 대해 복수의 박스를 발견했을 가능성이 높음.
NMS는 같은 물체에 대한 중복된 박스를 탐지하고 통합해서 물체 하나마다 하나의 박스만 남도록 하는 역할을 함
 - **평가 지표** : 이미지 분류의 정확도, 정밀도, 재현율 등과 비슷하게 사물 탐지에도 성능을 측정하는 고유의 평가 지표가 있음.
그 중 가장 널리 쓰이는 **평균평균정밀도** mean Average Precision, **mAP** 와 **PR 곡선** Precision-recall curve, **중첩률** Intersection over Union, **IoU** 을 설명함

1. 사물 탐지 알고리즘

영역 제안 Region Proposal

- 시스템 AI모델 이 이미지를 관찰하고 추가 분석이 필요한 관심 영역 Roi 를 제안하는 단계
- Roi는 시스템이 이미지의 해당 위치에 높은 확률(물체 존재 확신도 objectness score)로 물체가 존재한다고 판단한 영역
- 물체 존재 확신도가 높은 영역은 다음 단계로 정보가 전달되고, 낮은 영역은 추가 분석 없이 해당 영역 정보 폐기



물체 존재 확신도가 낮음
(배경)

물체 존재 확신도가 높음
(전경)

1. 사물 탐지 알고리즘

영역 제안 Region Proposal

- <영역 제안> 방식은 여러 가지가 있음
- 초기에는 **선택적 탐지** selective search 알고리즘을 사용하였음
- 각 시스템 AI모델 이 영역 제안을 해결하는 서로 다른 방법에 대해서는 나중에 다룸
- 지금은 이 단계에서 생성된 많은 수(수천 개에 이르기도 함)의 후보 영역이

이후 단계에서 분석 및 분류를 거치게 된다는 것만 기억하길 바람

- 신경망은 이미지를 물체 존재 확신도에 따라 **전경(물체)** Foreground 과 **배경(물체 아님)**으로 나뉘며, 이때 설정된 **임계값** threshold 을 초과하는 영역을 전경으로

간주하고 신경망의 이후 층에서 추가 분석의 대상이 됨

1. 사물 탐지 알고리즘

영역 제안 ^{Region Proposal}

- 임계값은 문제에 따라(데이터셋 에 따라) 조절
- 임계값이 너무 낮게 설정되면 거의 가능한 모든 영역을 제안하기 때문에 (장점) 이미지
에 있는 물체를 놓칠 가능성이 낮지만, (단점)계산 자원을 지나치게 소모하므로
사물 탐지 자체가 느려지는 단점이 있음
- 영역 제안 수와 계산 복잡도의 트레이드오프^{tradeoff} 관계를 잘 파악하고 문제에
적합한 수의 Roi를 제안하도록 설정 해야함

1. 사물 탐지 알고리즘

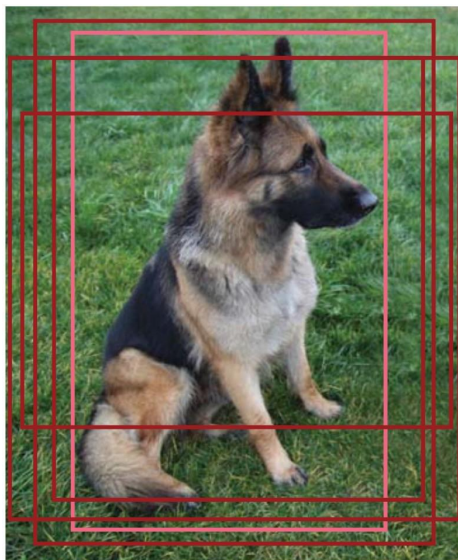
특징 추출 및 예측

- 이 모듈에는 특징 추출을 위해 **사전 학습된 합성곱 신경망**이 포함되며, 일반화 성능이 높은 이미지 분류 모델을 선택해서 사용함
- 모듈에서 추출된 특징을 이용해 물체의 유형을 분류하는데 사용함
- MS COCO나 ImageNet 데이터 셋을 학습한 사전 학습 모델은 대부분 물체의 특징을 잘 추출할 수 있음 (즉, 일반화 성능이 좋음)
- 이 단계에서는 물체를 포함했을 가능성이 높다고 판단된 모든 영역(영역 제안 단계를 통과한 영역)을 신경망이 분석하며 각 영역마다 다음 두 가지를 예측함
 - **경계 박스** Bounding Box, BB **예측** : 물체를 감싸는 박스의 좌표를 예측함. 경계 박스의 좌표는 튜플(x,y,w,h)로 표현됨. x와 y는 경계 박스의 중심점 좌표이고, w와 h는 경계 박스의 너비와 높이임
 - **클래스 예측** : 해당 영역의 물체가 각 클래스에 속할 확률을 예측하는 소프트맥스 함수

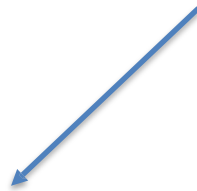
1. 사물 탐지 알고리즘

특징 추출 및 예측

- 앞단에서 제안되는 영역의 수가 많기 때문에 대부분 물체마다 여러 경계 박스가 생김
- 아래의 그림과 같이 개 이미지를 보면, 신경망이 물체를 잘 찾아낸 것을 볼 수 있음
- 그러나 이전 단계에서 개가 포함된 Roi가 5개나 제안되었기 때문에 개 주변에 경계 박스도 5개가 표시됨. 물체의 위치와 종류가 모두 정확하게 파악됐지만, 이 상태로는 우리가 원하는 결과가 되지 못함



사물 1개에 여러 개의 경계 박스를 예측함. 같은 사물을 포함하는 여러 개의 경계 박스를 사물을 가장 잘 포함하는 박스로 통합 필요



1. 사물 탐지 알고리즘

- 비최대 억제^{non-maximum suppression, NMS}

- 사물 탐지 알고리즘의 문제점 중 하나는 같은 물체를 여러 번 탐지한다는 것이며, 이 때문에 물체 하나에 경계 박스가 여러 개 표시될 수 있음
- 비최대 억제는 물체 하나에 경계 박스가 하나만 남도록 하는 기법
- 이름에서 짐작할 수 있듯이 **예측 확률이** **predicted probability** 가장 높은 경계 박스만 남기고 나머지는 **배제**하는 방식



Predictions before NMS



After applying non-maximum suppression

1. 사물 탐지 알고리즘

비최대 억제^{non-maximum suppression, NMS}

- NMS 알고리즘의 작동 과정은 다음과 같음

- 1) 예측 확률^{predicted probability}이 미리 설정된 신뢰 임계값^{confidence threshold}에 미치지 못하는 경계 박스를 폐기 (신뢰 임계 값은 수정 가능)
- 2) 남아 있는 경계 박스를 하나씩 살펴본 후 예측 확률이 가장 높은 것을 선택
- 3) 선택된 박스와 예측 클래스가 같은 박스의 중복 영역을 계산.

이 지표를 **중첩률**^{Intersection over union, IOU}이라고 함.

- 4) 중첩률이 0인 박스는 남겨두고, 중첩률이 미리 설정한 임계값(NMS 임계값)보다 큰 경우 경계 박스에서 배제. (중첩률이 높다는 뜻은 같은 물체를 검출하고 있다는 판단 증거이므로)

1. 사물 탐지 알고리즘

사물 탐지 성능 평가 지표

- 사물 탐지의 성능을 평가하는 지표는 크게 초당 프레임 수 frames per second, FPS 와

평균평균정밀도 mean Average Precision, mAP 두 가지가 쓰임

초당 프레임수: 탐지 속도의 평가 지표

- 탐지 속도를 평가하는 일반적인 지표로는 초당 프레임 수^{FPS}가 사용됨
- 논문에서 성능 비교 실험 결과는 “신경망 X는 Z FPS의 속도에서 mAP Y%를 달성했다”라고 언급되며, 여기서 X는 신경망의 종류이고, Y는 평균평균정밀도, Z는 초당 프레임 수에 해당됨

평균평균정밀도: 신경망 예측 정밀도의 평가 지표

- 물체 인식에서 가장 널리 쓰이는 성능 평가 지표는 평균평균정밀도^{mAP}가 사용됨
- 0에서 100 사이의 값으로 나타내며 값이 클수록 성능이 좋은 것은 정확도와 동일
- mAP 계산 방법을 이해하려면 먼저 IoU와 PR 곡선을 이해해야 함

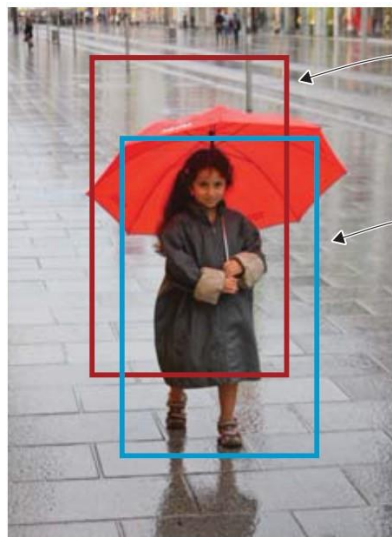
1. 사물 탐지 알고리즘

중첩률(IoU, Intersection over Union, 교집합/합집합 비율)

- 2개의 경계 박스가 중첩되는 정도를 나타내는 값
- 정답 경계 박스($BB^{\text{ground truth}}$)와 예측 경계 박스($BB^{\text{predicted}}$)가 있다고 할 때

IoU를 계산해서 해당 탐지 결과가 유효^{true positive, TP}한지 아닌지^{false positive, FP}

결정할 수 있음



예측 경계
박스(사람)

정답 경계
박스(사람)

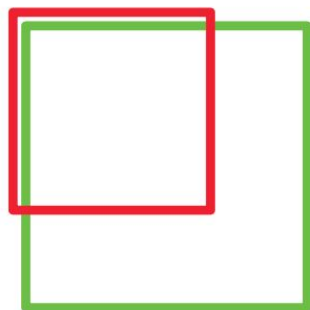
$$\text{중첩률} = \frac{\text{중첩되는 부분의 면적}}{\text{두 박스를 합친 부분의 면적}}$$

1. 사물 탐지 알고리즘

중첩률(IoU, Intersection over Union, 교집합/합집합 비율)

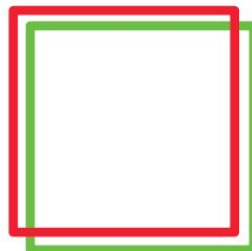
- 중첩률은 0(두 경계 박스가 전혀 겹치지 않음)부터 1(두 경계박스가 완전히 겹침)까지의 값을 가지며 값이 클수록 좋음
- 중첩률은 정확한 예측을 정의하는 용도로 사용됨
 - 중첩률이 설정된 임계값보다 크면 정확한 예측이 됨을 의미함
 - 이 임계값은 상황에 따라 조정 가능하지만 일반적으로 0.5가 많이 사용됨
- MS COCO 경진대회에서는 mAP@0.5 (중첩률 임계값 0.5) 또는 mAP@0.75 (중첩률 임계값 0.75)이 쓰임

IoU: 0.4034



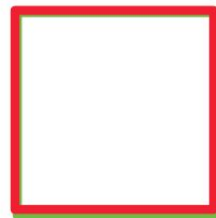
Poor

IoU: 0.7330



Good

IoU: 0.9264



Excellent

1. 사물 탐지 알고리즘

		실제 정답	
		True	False
분류 결과	True	True Positive	False Positive
	False	False Negative	True Negative

- True Positive(TP) : 실제 True인 정답을 True라고 예측 (정답)
- False Positive(FP) : 실제 False인 정답을 True라고 예측 (오답)
- False Negative(FN) : 실제 True인 정답을 False라고 예측 (오답)
- True Negative(TN) : 실제 False인 정답을 False라고 예측 (정답)

1. 사물 탐지 알고리즘

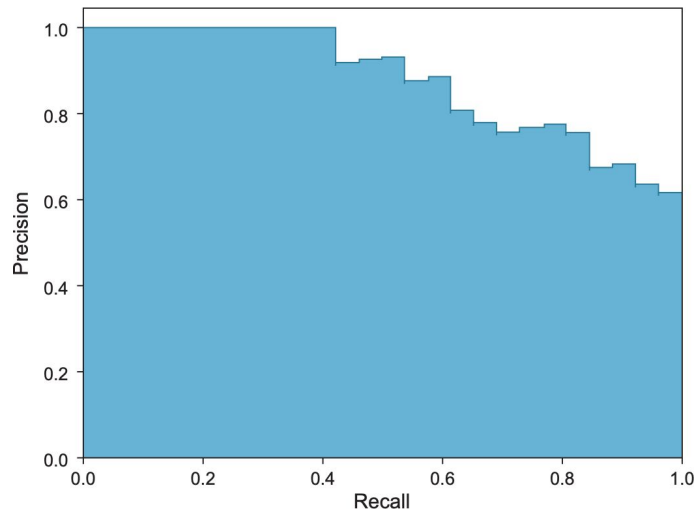
PR 곡선

- 진양성^{TP}과 위양성^{FP}를 정의했으니 이제 주어진 클래스에 대한 탐지 결과의 정밀도와 재현율을 계산할 수 있음

$$(Recall) = \frac{TP}{TP + FN}$$

$$(Precision) = \frac{TP}{TP + FP}$$

- 모든 대상 클래스에 대한 정밀도와 재현율을 계산하고 나면, 다음과 같은 PR 곡선을 그릴 수 있음



1. 사물 탐지 알고리즘

PR 곡선

- 각 클래스마다 **PR 곡선**을 그려보면 사물 탐지 모델의 성능을 평가 가능
- 재현율이 증가할 때에도 정밀도가 하락하지 않는다면 임계값을 변화시켜도 정밀도가 하락하지 않는다는 의미이므로 모델의 성능이 뛰어나다는 의미
- 반대로 성능이 낮은 모델은 재현율을 끌어올리면서 위양성^{FP}의 수가 늘어나고, 대부분 PR곡선이 높은 정밀도로 시작해서 정밀도가 하락하는 패턴을 보임
- PR곡선 아래의 면적인 **AUC**^{area under the curve}를 계산하여 **평균정밀도**^{AP}를 계산할 수 있고, 각 클래스의 평균정밀도의 평균인 **평균평균정밀도**^{mAP}를 계산하여 최종적으로 <사물 탐지 모델의 성능>을 평가 할 수 있음

1. 사물 탐지 알고리즘

정리

mAP 계산법을 정리하면 다음과 같음

- (1) 각 경계 박스와 물체 존재 확신도 해당 경계 박스 영역 안에 물체가 존재할 확률 **를 계산**
- (2) 정밀도와 재현율을 계산
- (3) 각 분류 클래스마다 확률의 임계값을 변화시키며 **PR곡선**을 그림
- (4) PR 곡선의 **AUC를 계산**하여 각 분류 클래스마다 평균정밀도를 계산
- (5) 각 클래스의 평균정밀도의 평균을 계산

2. 영역 기반 합성곱 신경망(R-CNN)

영역 기반 합성곱 신경망

- 영역 기반 합성곱 신경망 region-based convolutional neural network, R-CNN 은 2014년에 Ross Girshick 이 제안한 신경망 구조
- R-CNN은 2015년에 Fast R-CNN, 2016년에 Faster R-CNN이 잇달아 제안되며 변종을 늘려 나갔음

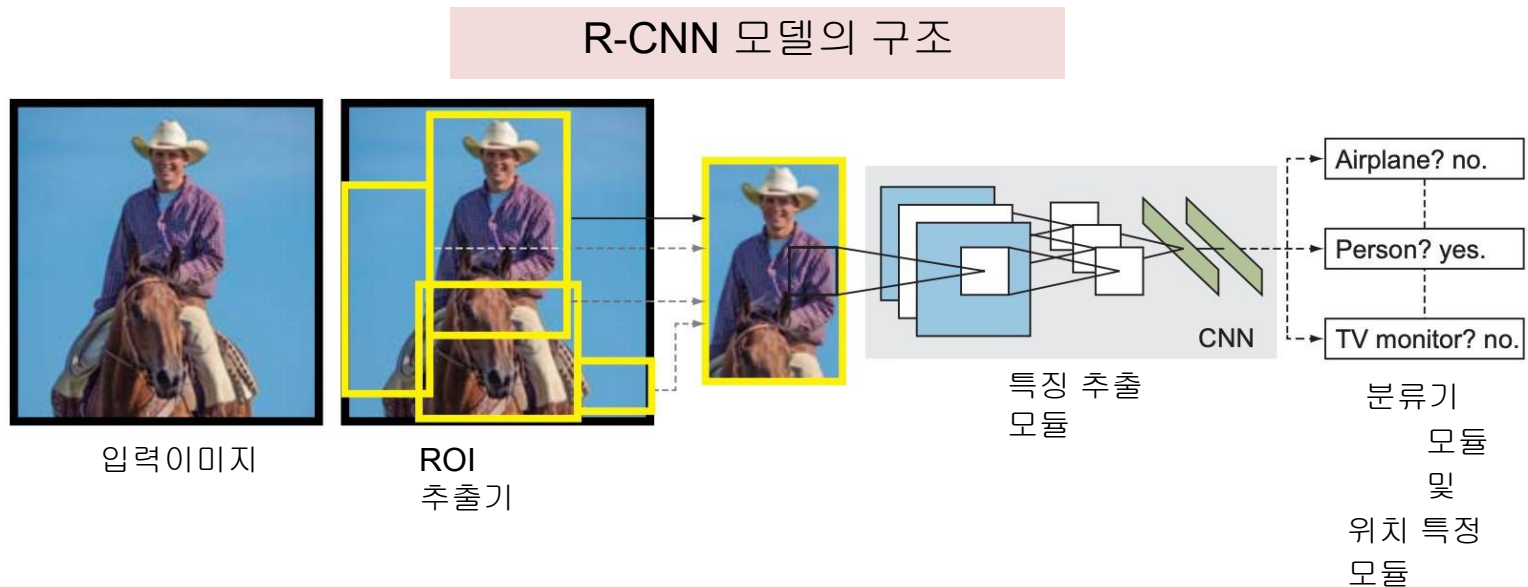
R-CNN 이란

- R-CNN 은 영역 기반 신경망 구조 중 가장 기본적인 구조지만, 모든 다중 물체 인식 알고리즘의 동작을 이해하기 위한 밑바탕이 되는 구조임
- R-CNN은 합성곱 신경망이 물체 인식 및 위치 특정 문제에서 대규모로 응용된 최초의 사례이며, 또한 이후 제안된 사물 탐지 알고리즘의 기반이 되기도 했음
- R-CNN이 제안한 기법은 PASCAL VOC-2012 데이터셋과 2013 ILSVRC에서 당시 최고 성능 기록을 갱신한 바 있음

2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 이란

- R-CNN의 네 가지 구성요소는 다음과 같음
 - Roi 추출기
 - 특징 추출 모듈
 - 분류 모듈
 - 위치 특정 모듈



2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 이란

- R-CNN의 네 가지 구성요소는 다음과 같음
 - **Roi 추출기**: 영역 제안 모듈이라고 한다.

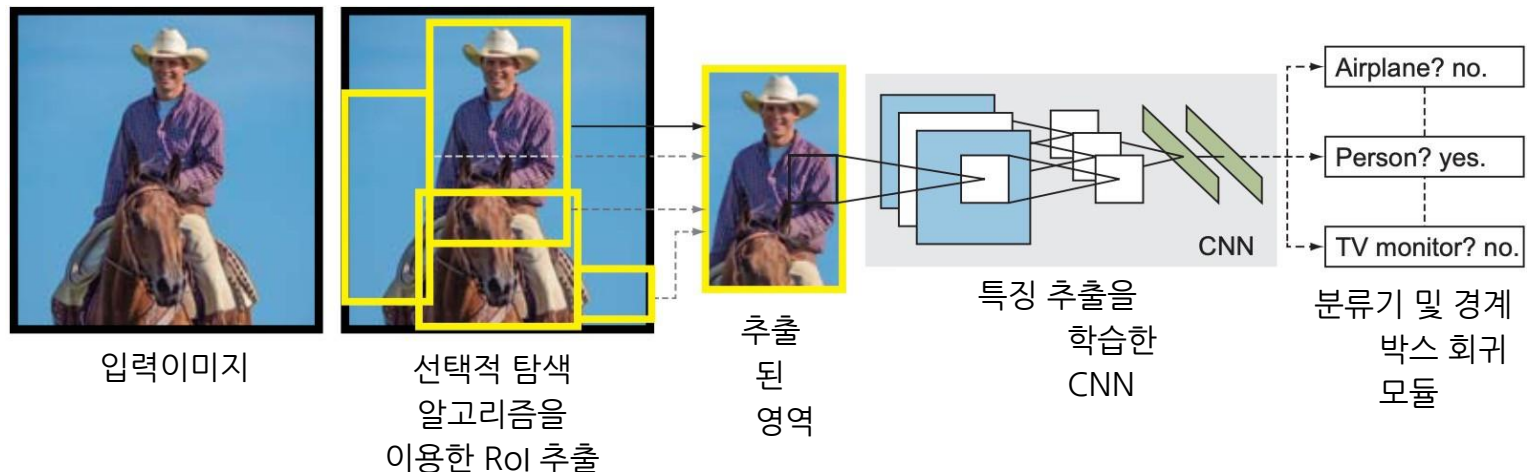
Roi는 물체를 포함하고 있을 가능성이 높은 이미지의 영역이다.

선택적 탐색 알고리즘을 이용해서 입력 이미지를 스캔하며 얼굴 패턴이 있는 영역을 찾아 다음 단계에서 추가 처리하도록 Roi로 제안한다.

제안된 Roi는 고정된 크기로 추출된다.

(Roi 영역의 크기는 일반적으로 제각각 이지만 CNN에 입력하려면 고정된 크기여야 하기 때문이다.)

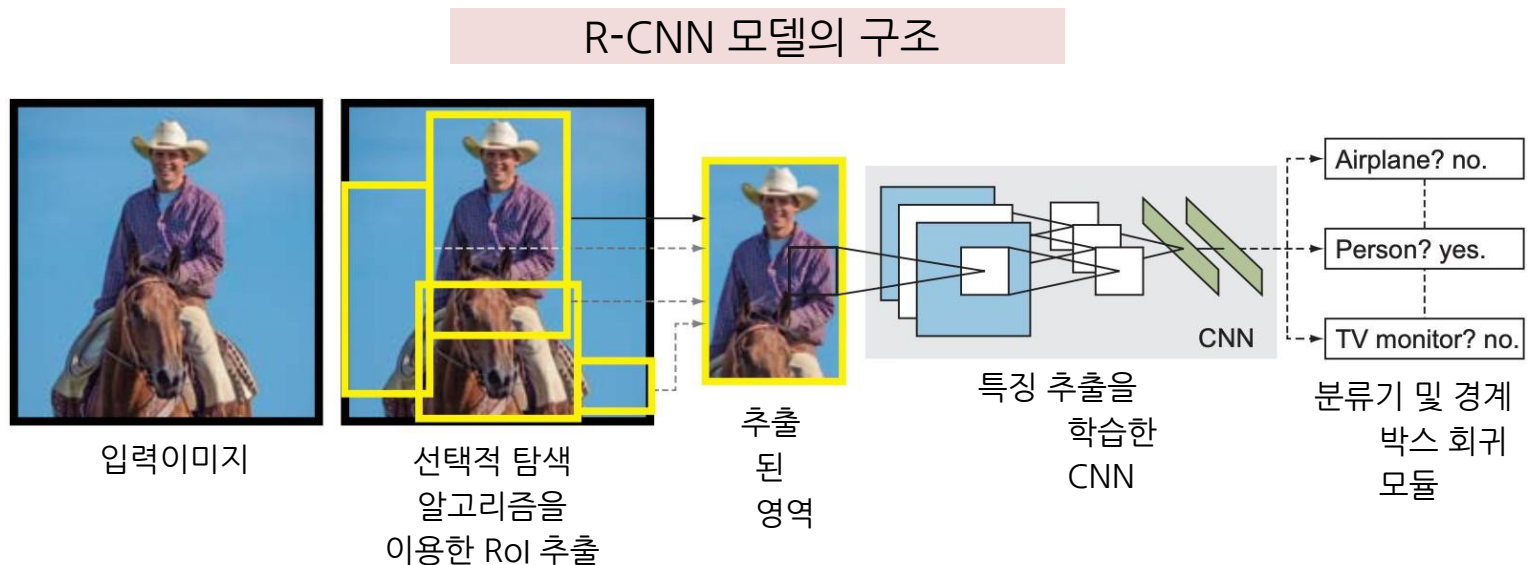
R-CNN 모델의 구조



2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 이란

- R-CNN의 네 가지 구성요소는 다음과 같음
 - **특징 추출 모듈**: 사전 학습된 합성곱 신경망에 앞서 제안된 Roi 영역을 입력해서 특징을 추출한다. 이 과정은 우리가 이전 장에서 배운 일반적인 합성곱 신경망 특징 추출기가 맡는다.

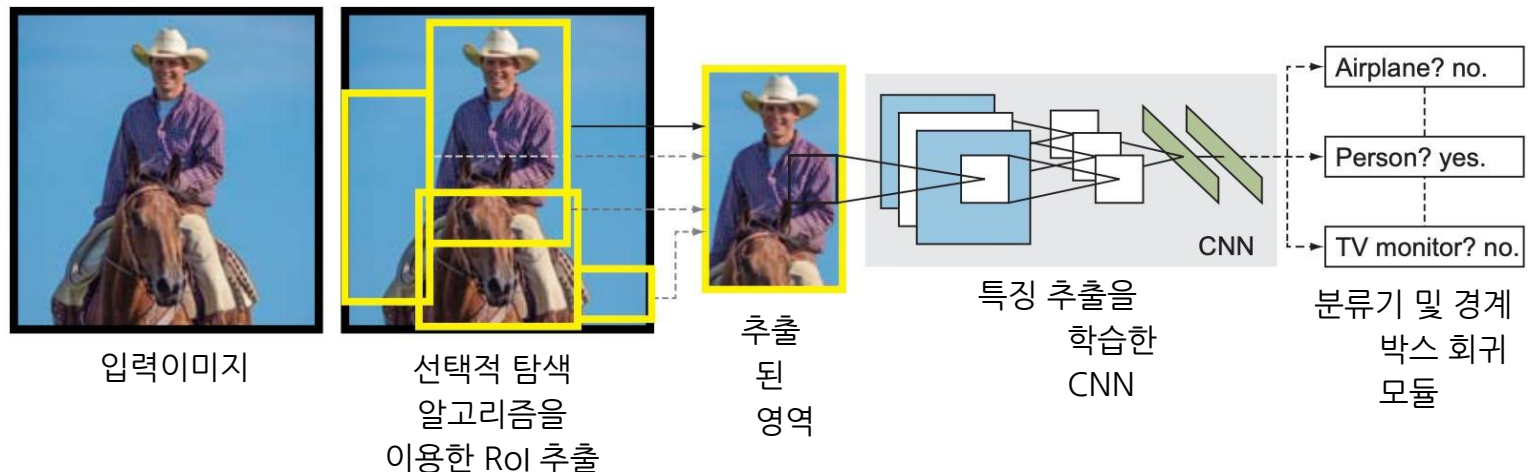


2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 이란

- R-CNN의 네 가지 구성요소는 다음과 같음
 - **분류 모듈**: 서포트 벡터 머신 등 기존 머신러닝 알고리즘으로 분류기를 학습한 다음 특징 추출 모듈에서 추출한 특징으로 해당 영역의 물체가 무엇인지 분류한다.
 - **위치 특정 모듈**: **경계 박스 회귀 모듈** **bounding-box regressor** 이라고 한다. 이 모듈의 예측 대상은 경계 박스의 위치와 크기다. 경계 박스는 (x,y,w,h) 와 같은 튜플로 표현되는데, x 와 y 는 경계 박스의 중심점 좌표고, w 와 h 는 경계 박스의 너비와 높이다.

R-CNN 모델의 구조

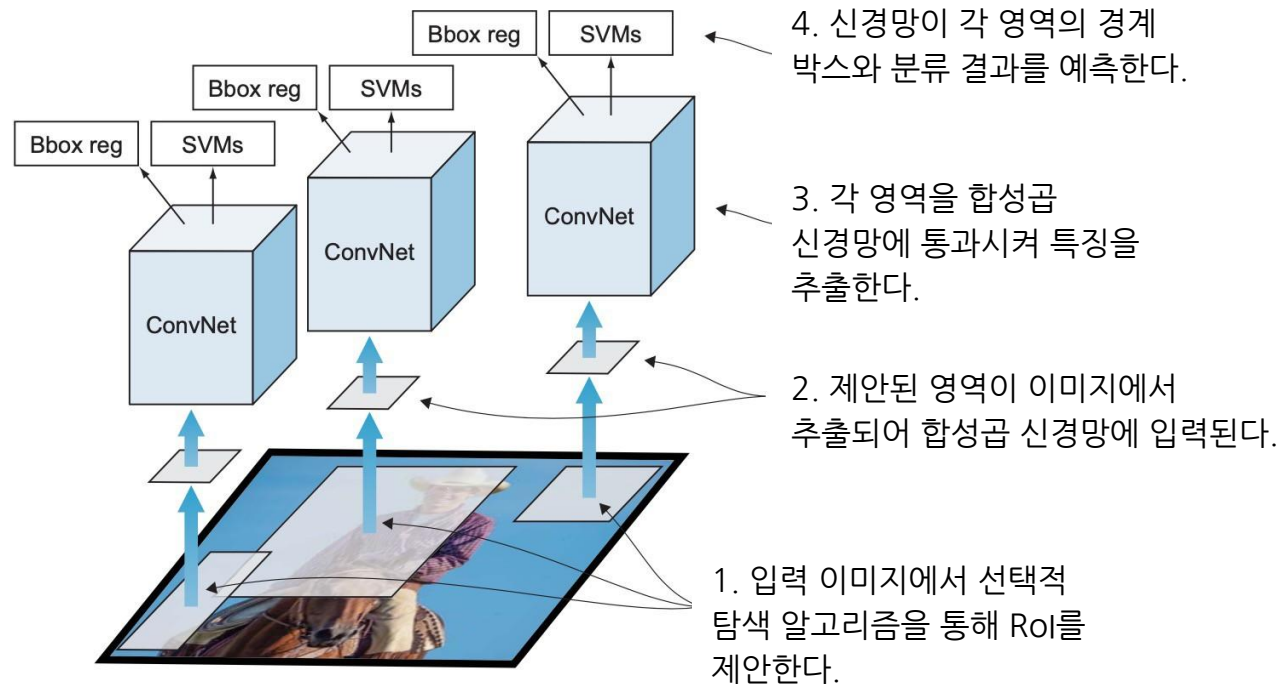


2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 이란

- 아래 그림은 R-CNN 신경망 구조를 그림으로 알기 쉽게 나타낸 것임
- 그림에서 볼 수 있듯이 먼저 Roi를 제안한 다음 각 영역에서 특징을 추출하고 이 특징을

바탕으로 영역을 분류함



2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 학습

- R-CNN은 Roi 추출기, 특징 추출 모듈, 분류 모듈, 경계 박스 회귀 모듈 이렇게 4개의 요소로 구성됨
- Roi 추출기를 제외한 모든 모듈은 학습*이 필요하며, R-CNN 학습을 위해 다음 절차를 따라야 함
 - 먼저 특징 추출 모듈로 사용할 합성곱 신경망을 학습한다. 일반적으로 합성곱 신경망의 학습 절차를 따르면 된다. 신경망을 처음부터 학습시켜도 되지만 이렇게 하는 경우는 드물고, 대개 사전 학습된 신경망을 미세 조정해서 사용한다.
 - SVM 분류기를 학습한다. SVM 알고리즘은 이 책에서 다루지 않지만 딥러닝을 이용한 분류기와 마찬가지로 레이블링된 데이터를 학습해야 한다.
 - 경계 박스 회귀 모듈을 학습한다. 이 모델의 출력은 K 가지 클래스에 속하는 물체가 포함된 경계 박스를 나타내는 4개의 실수 값이다.

2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN 학습

- 위 학습 절차를 살펴보면 R-CNN의 학습에 필요한 비용과 시간이 그리 만만치 않음을 알게 됨
- 3개의 모듈을 완전히 별개로 학습시켜야 하고 이미 계산한 결과를 재사용할 수 없음
- 이처럼 다단계 파이프라인으로 구성된 학습 과정은 R-CNN의 주요 단점으로 꼽힘

R-CNN의 단점

- R-CNN은 구조를 이해하기 쉽고, 최초 제안되었던 당시에는 최고 성능을 달성했음
- 특히 합성곱 신경망을 사용해서 특징을 추출한 경우 더욱 성능이 좋았음
- 그러나 R-CNN은 하나의 신경망으로 구성된 시스템이 아니며 서로 독립된 여러 알고리즘이 조합되어 동작하는 시스템임

2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN의 단점

- 이런 구조로 인해 다음과 같은 단점이 존재함
 - **사물 탐지 속도가 느리다.** 이미지 한 장마다 2000개 이상의 Roi가 제안되고 이들 영역이 모두 전체 파이프라인(CNN 특징 추출기 및 분류기)을 통과해야 하는 구조다. Roi 하나마다 합성곱 신경망의 순방향 계산이 필요하며 그 결과를 재사용하지 못하므로 계산 부하가 매우 크고 속도가 느리다. 계산 부하가 크다는 점이 R-CNN이 널리 응용되지 못하는 원인이 되었다. 특히 자율주행차 등 실시간 처리가 필요한 분야에서 큰 문제가 되었다.
 - **학습 과정이 다단계로 구성된다.** 앞서 설명했듯이 R-CNN을 학습하려면 CNN 특징 추출기, SVM분류기, 경계 박스 회귀 모듈까지 세 모듈을 학습해야 한다. 이들 모듈의 학습 과정은 복잡하고 각각 따로 진행해야 한다.
 - **학습의 공간 및 시간 복잡도가 높다.** SVM 분류기와 경계 박스 회귀 모듈을 학습할 때 각 Roi에서 추출된 특징을 디스크에 저장해야 한다. VGG16 같은 대규모 신경망을 사용하면 수천 장의 이미지를 학습하는데 GPU를 사용해도 수일이 걸린다. 추출된 특징을 저장하기 위해 필요한 하드 디스크 용량은 수백 GB나 된다.

2. 영역 기반 합성곱 신경망(R-CNN)

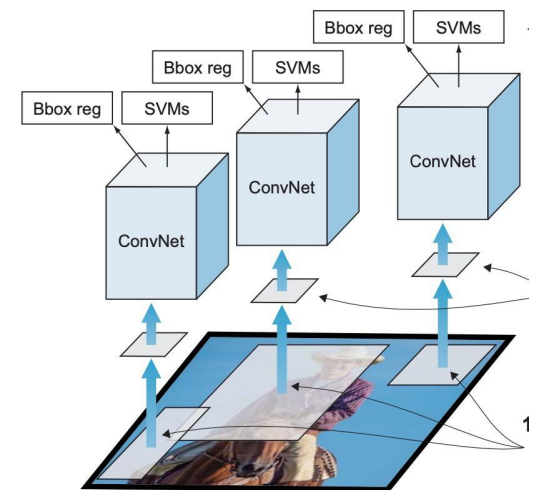
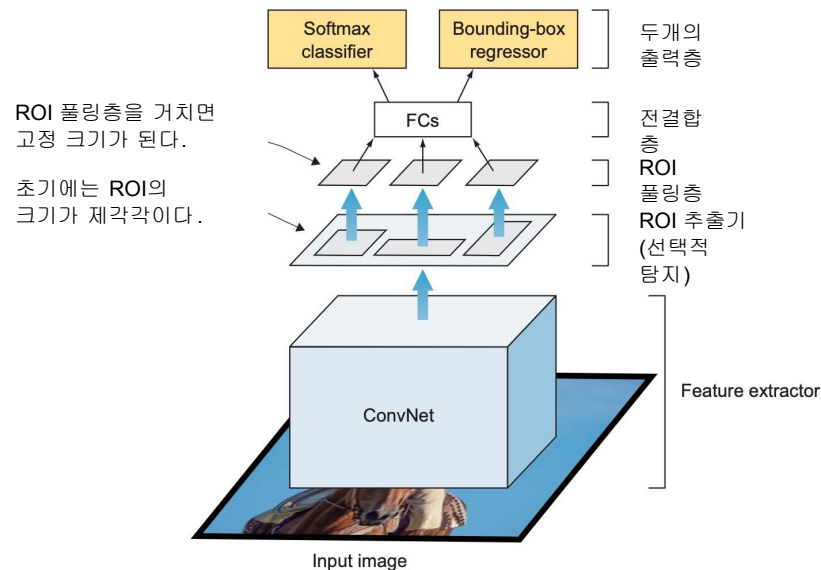
Fast R-CNN

- Fast R-CNN은 2015년 로스 거식이 제안한 R-CNN의 변종으로, R-CNN의 직계 후손이라고 할 수 있음
- Fast R-CNN은 여러 가지 측면에서 R-CNN과 비슷한 점이 많지만 다음 두 가지 설계 수정을 통해 처리 속도와 정확도가 개선되었음
 - 영역 제안 모듈로 파이프라인을 시작해서 그 뒤로 특징 추출 모듈을 배치하는 대신 CNN 특징 추출기를 맨 앞에 배치해 전체 입력 이미지로부터 영역을 제안한다. 이렇게 하면 2000개 이상의 중첩된 영역을 2000개 이상의 CNN으로 처리하는 대신 전체 이미지를 하나의 CNN으로 처리할 수 있다.
 - CNN에 SVM 분류기 대신 소프트맥스층을 추가해서 분류도 수행하게 된다. 이렇게 하면 하나의 모듈이 두가지 일(특징 추출, 사물 분류)을 할 수 있게 된다.

2. 영역 기반 합성곱 신경망(R-CNN)

Fast R-CNN의 구조

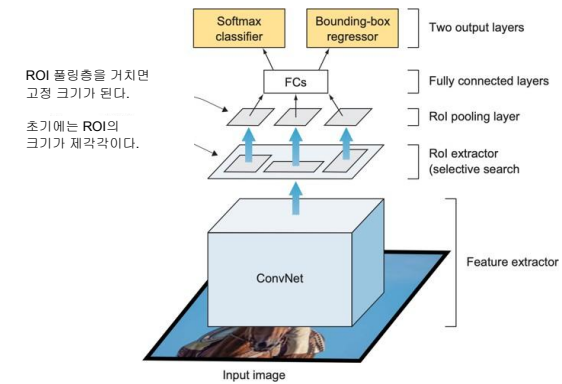
- Fast R-CNN은 신경망의 마지막 특징맵을 바탕으로 영역을 제안함
- 이런 설계 덕분에 전체 이미지를 입력 받는 **CNN 하나만 학습**하면 됨
- 물체의 각 클래스를 대상으로 하는 SVM 분류기를 여러 개 학습하지 않고 소프트맥스층을 두어 각 클래스의 확률을 직접 구함
- 신경망 하나와 여러 개의 SVM 분류기를 학습시켜야 했던 복잡한 학습 과정이 신경망 하나로 매우 간단해짐



2. 영역 기반 합성곱 신경망(R-CNN)

Fast R-CNN의 구조

- Fast R-CNN의 구성요소
 - 특징 추출 모듈: 신경망은 ConvNet 으로 시작하며 전체 이미지에서 특징을 추출한다.
 - Roi 추출기: 입력 영상에서 선택적 탐지 알고리즘을 사용해서 약 2000개 영역을 뽑는다.
 - Roi 풀링층: Fast-RCNN 에서 새로 추가된 요소로, 특징 맵에서 전결합층에 입력할 고정 크기의 영역을 추출하는 역할을 한다. 제안된 Roi 영역에 해당하는 특징을 최대 풀링을 통해 고정 크기 (HxW)로 추출한다.
- 2개의 출력층: 이 모델은 다음 2개의 출력층을 갖는다.
 - Roi의 각 클래스에 속할 확률을 출력하는 소프트맥스층
 - 최초 제안된 Roi와의 차이를 출력하는 경계 박스 회귀층



2. 영역 기반 합성곱 신경망(R-CNN)

Fast R-CNN의 다중 과업 손실 함수

- Fast R-CNN은 물체의 클래스 분류와 경계 박스 위치 및 크기를 함께 학습하는 완결된 신경망 구조임
- 여기에 사용되는 손실 함수도 클래스 분류와 경계 박스 위치 및 크기를 함께 다루는 다중 과업 손실 multi-task loss 함수 여야 함
- 모든 최적화 문제에는 최적화 대상이 될 손실 함수가 필요함
- 사물 탐지 문제는 사물 분류와 물체 위치 특정 두 가지 목표를 최적화하는 것이 목표임
- 손실 함수도 2개가 있어야 하며, L_{cls} 는 사물 분류의 손실 함수 이고, L_{loc} 는 물체 위치 특정의 손실 함수임

2. 영역 기반 합성곱 신경망(R-CNN)

Faster R-CNN

- Faster R-CNN 은 세 번째로 고안된 R-CNN의 변종으로 2016년 샤오칭 렌이 제안
- Fast R-CNN과 마찬가지로 이미지가 합성곱 신경망에 그대로 입력되어 특징 맵을 생성
- ① 영역 제안을 위해 선택적 탐지 알고리즘 대신 **영역 제안 신경망** region proposal network, RPN 을 도입해서 영역 제안을 학습 과정에 포함
- ② 예측 영역 제안은 RoI 풀링층을 사용하여 재구성되고 제안된 영역 내에서 이미지를 분류하고 경계 박스의 오프셋값 예측에 사용
- 두 가지 개선 사항(①, ②) 모두 영역 제안 수를 감소시켜 성능 측정 시점의 연산량을 줄이고 실시간에 가까운 처리를 수행하며 당시 최고 성능을 달성함

2. 영역 기반 합성곱 신경망(R-CNN)

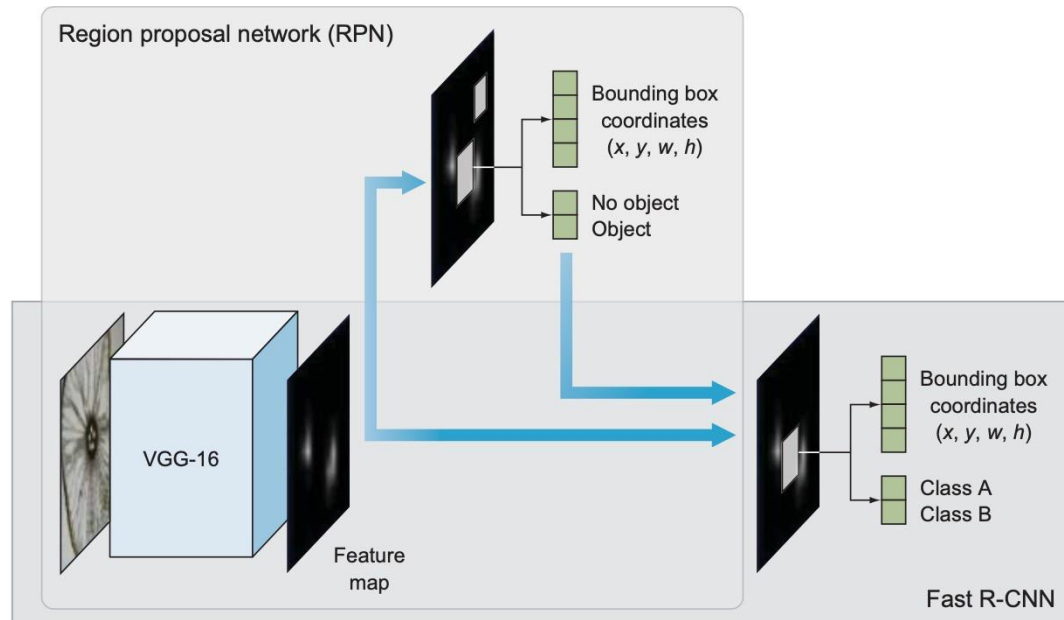
Faster R-CNN 의 구조

- Faster R-CNN의 구조는 다음 두 신경망으로 구성됨
 - **영역 제안 신경망^{RPN}** : 선택적 탐색 알고리즘을 대체하는 합성곱 신경망
 - 특징 추출기에서 생성한 특징 맵에서 추가 분석을 이어갈 RoI를 제안하는 역할을 수행
 - RPN의 출력은 물체 존재 확신도와 경계 박스 위치
 - **Fast R-CNN** : Fast R-CNN의 일반적인 구성 요소로 구성
 - 특징 추출기 역할을 하는 기본 신경망: 입력 이미지에서 특징을 추출하는 일반적인 사전 학습 CNN 모델
 - RoI를 고정 크기로 변환하는 RoI 풀링층
 - 2개의 전결합층으로 나뉜 출력층. 하나는 클래스 분류 확률을 출력하는 소프트맥스 분류기이고, 다른 하나는 경계 박스를 예측하는 경계 박스 회귀 모듈 CNN

2. 영역 기반 합성곱 신경망(R-CNN)

Faster R-CNN 의 구조

- 입력 이미지에서 사전 학습된 CNN을 이용해서 특징을 추출
- 이 특징은 병렬로 Faster R-CNN의 다음 두 갈래 경로를 따라 전달됨
 - RPN이 이미지 내 어느 위치에 물체가 존재하는지 판단하는 경로. 이미지에 있는 물체가 무엇인지 알 수 없으며, 어떤 위치에 물체가 있을 수도 있다는 가능성이 있을 뿐임
 - RoI 풀링층이 고정 크기 특징을 추출하는 경로



2. 영역 기반 합성곱 신경망(R-CNN)

Faster R-CNN 의 구조

이들 출력은 각각 2개의 전결합층으로 전달됨

- 하나는 <물체를 분류>하는 층
- 다른 하나는 <물체의 위치>를 최종적으로 특정하기 위한 경계 박스를 예측하는 층

이 신경망 구조는 전체 구조가 학습 가능하며, 다음과 같은 요소로 구성된 사물 탐지 파이프라인 전체를 신경망에 내장시키는데 성공함 (end-to-end model)

- 기본 특징 추출기
- 영역 제안 모듈
- RoI 풀링층
- 사물 분류기
- 경계 박스 회귀 모듈

2. 영역 기반 합성곱 신경망(R-CNN)

기본 특징 추출기

- Fast R-CNN과 비슷하게 먼저 사전 학습된 CNN에서 분류기 부분을 잘라 사용함
- 기본 특징 추출기는 입력 이미지에서 특징을 추출하는데 사용됨
- 해결하려는 문제에 적합한 CNN 구조라면 어떤 것이라도 특징 추출기로 사용할 수 있음
- Faster R-CNN의 논문에서는 이미지넷 데이터셋으로 사전 학습된 **ZF**와 **VGG** 신경망을 사용하였으나 그 후로 가중치 수가 제각각인 다양한 신경망이 제안되었음
- 예를 들어, **MobileNet**은 효율적으로 규모를 줄여 속도에 최적화된 것이 특징으로 330만 개의 파라미터를 가짐
- 그 반대쪽 끝에는 이미지넷 이미지 분류 경진대회에서 최고 성능을 갱신한 바 있는 ResNet-152가 있으며, 이 신경망 가중치 수는 6천만 개에 달함
- 최근에 제안된 신경망 구조인 **DenseNet**은 파라미터 수 감축과 성능 개선이라는 두 마리 토끼를 다 잡았음

2. 영역 기반 합성곱 신경망(R-CNN)

기본 특징 추출기

- 앞 장에서 설명했듯이 합성곱층은 자신의 이전 층에서 전달된 정보를 더욱 추상화하는 효과가 있음
- 첫 번째 층은 대개 모서리를 학습하며, 두 번째 층은 모서리가 조합된 복잡한 도형을 학습하는 식임
- 이런 식으로 마지막 층에서 RPN에서 입력해서 물체가 있는 영역을 추출할 수 있는 특징 맵을 만들어 냄

VGGNet과 ResNet

기존에 VGG가 널리 쓰였던 기본 특징 추출기의 역할을 최근에는 ResNet이 대체해가는 추세다. VGG와 비교한 ResNet의 장점은 층수가 많으므로 더 복잡한 특징을 학습할 수 있다는 것이다. 분류에서도 이 점은 동일하며, 사물 탐지에서도 마찬가지다. 잔차 연결과 배치 정규화를 적용한 ResNet은 층수가 많은 신경망을 학습하는데 더욱 유리하다.

2. 영역 기반 합성곱 신경망(R-CNN)

영역 제안 모듈(RPN)

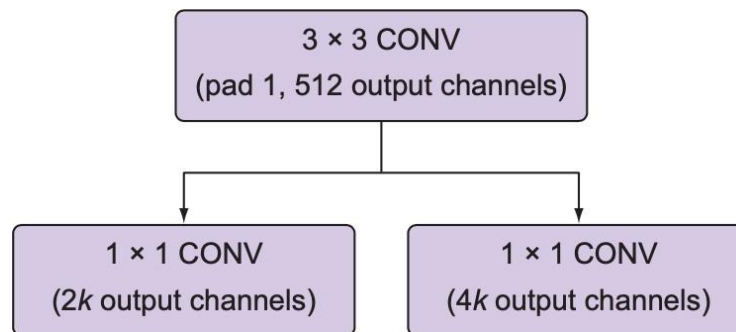
- RPN은 사전 학습된 합성곱 신경망에서 만든 특징 맵의 정보를 기초로 물체가 위치했을 법한 영역을 찾아내는 역할을 함
- RPN을 어텐션 네트워크^{attention network}라고 부르기도 하는데 이는 해당 신경망이 전체 신경망의 관심사를 이미지 내 물체가 있을 만한 영역으로 유도하는 역할을 하기 때문임
- Faster R-CNN은 선택적 탐색 알고리즘 대신 RPN을 사용해서 RoI 제안 기능을 신경망 구조 안에 포함시켰음

2. 영역 기반 합성곱 신경망(R-CNN)

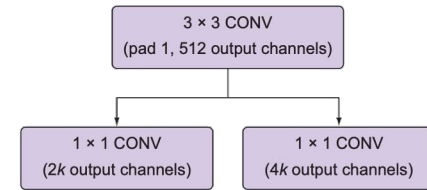
영역 제안 모듈(RPN)

- RPN은 다음 2개의 층으로 구성됨
 - 3x3 전결합 합성곱층 (512채널)
 - 병렬로 배치된 2개의 1x1 합성곱층
 - 하나는 해당 영역이 물체를 포함하는지 예측하는 분류층 (배경인지 전경인지 판단하는 점수)
 - 다른 하나는 경계 박스 회귀를 담당

합성곱층으로 구현한 RPN, 여기서 K는 앵커수



2. 영역 기반 합성곱 신경망(R-CNN)

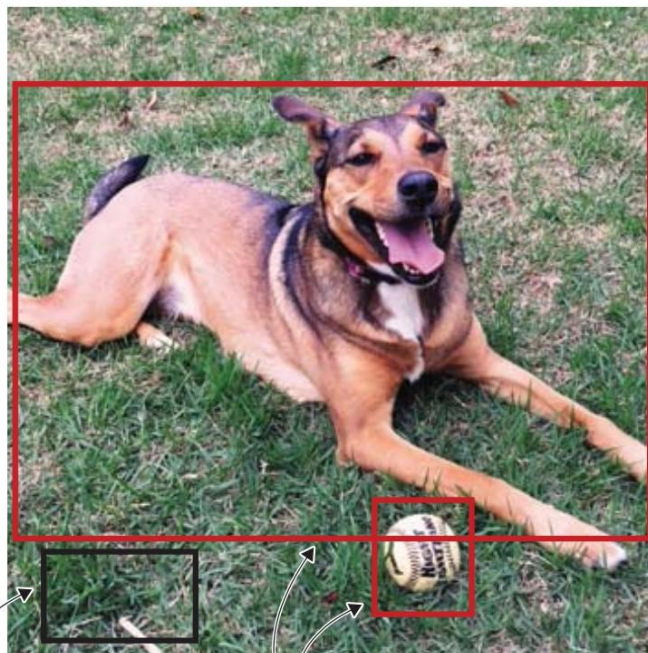


영역 제안 모듈(RPN)

- 3x3 크기의 슬라이딩 윈도우를 가진 기반 신경망의 마지막 특징 맵에 3x3 합성곱 연산이 적용됨
- 합성곱 연산 결과는 다시 2개의 1x1 합성곱층으로 전달됨
- 이 두 합성곱층은 각각 영역의 물체 포함 여부 예측과 경계 박스 회귀를 맡음
- 이들 두 층의 예측 목표는 물체의 클래스나 경계 박스가 아님
- 물체의 클래스나 경계 박스는 RPN 이후 좀 더 나중 단계에서 예측함
- 다시 한번 강조하지만 RPN의 목표는 해당 영역에 추가 분석이 필요한 물체가 존재하는지 여부를 판단하는 것임
- 그래서 RPN은 현재 영역의 물체 존재 확신도, 즉 해당 영역이 전경에 해당하는지 또는 배경에 해당하는지 그 확률을 판단하는 이진 분류기를 포함함
- 추가 분석이 필요한 물체가 존재한다고 판단되면 이 영역을 RoI 풀링층으로 전달함

2. 영역 기반 합성곱 신경망(R-CNN)

영역 제안 모듈(RPN)



물체 존재 확신도
낮음 (배경)

물체 존재 확신도
높음 (전경)

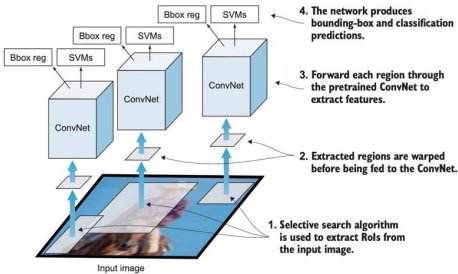
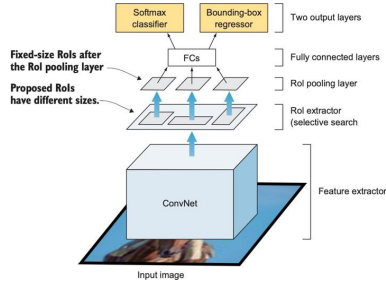
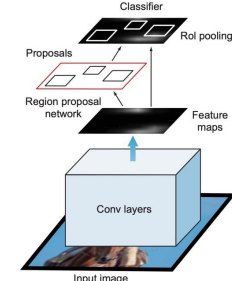
2. 영역 기반 합성곱 신경망(R-CNN)

전결합 합성곱 신경망(Fully Convolution Network-FCN)

- 사물 탐지에 사용되는 신경망의 중요한 특징 중 하나는 FCN을 포함한다는 점임
- FCN에는 보통 신경망 마지막 층에 배치되어 예측 결과를 만드는 전결합층이 없음
- 이미지 분류에서 전결합층을 사용하지 않는 방법은 보통 최종 예측 결과를 내놓는 전결합 소프트맥스 층 대신 평균 풀링을 적용하는 것임
- FCN의 장점은 다음과 같음
 - 전결합층 없이 합성곱 연산만 사용하므로 속도가 빠름
 - 이미지와 신경망을 읽어 들일 메모리만 충분하다면 받아드릴 수 있는 이미지의 크기에 제한이 없음
- FCN은 입력 이미지의 크기가 고정된다는 단점이 있지만 사실 알고리즘 구현상의 문제로 고정 크기 입력을 사용하는 것이 더 편한 경우가 많음
 - GPU를 사용하여 배치 단위로 이미지를 처리하는 경우가 이에 해당함
 - 이런 경우 모든 이미지의 가로세로 크기가 동일해야함

2. 영역 기반 합성곱 신경망(R-CNN)

■ R-CNN과 그 변종 정리

	R-CNN	Fast R-CNN	Faster R-CNN
	 1. Selective search algorithm is used to extract RoIs from the input image. 2. Extracted regions are warped before being fed to the ConvNet. 3. Forward each region through the pretrained ConvNet to extract features. 4. The network produces bounding-box and classification predictions.	 Input image ConvNet Feature extractor RoI extractor (selective search) RoI pooling layer Fully connected layers Two output layers Softmax classifier Bounding-box regressor Fixed-size RoIs after the RoI pooling layer Proposed RoIs have different sizes.	 Input image Conv layers Feature maps Region proposal network Proposals RoI pooling Classifier
정확도*	66.0%	66.9%	66.9%
특징	- 선택적 탐색 알고리즘을 사용해서 RoI (2000개 이상)를 추출함. - 별도의 합성곱 신경망을 사용해서 각 RoI에서 특징을 추출함. - 경계 박스 예측과 클래스 분류를 함께 수행함	각 이미지는 CNN에 한 번만 전달되어 특징 맵이 추출된다. - CNN을 사용해서 입력 이미지에서 특징 맵을 추출함. - 특징 맵에 선택적 탐색 알고리즘을 적용함. 이런 방식으로 2000곳 이상의 RoI를 모두 합성곱 신경망으로 처리하는 대신 전체 이미지를 한번만 처리하면 된다.	선택적 탐색 알고리즘을 RPN으로 대체해서 속도를 개선했다. 전체 처리 과정을 처음부터 끝까지 딥러닝으로 구현했다.
단점	RoI를 하나씩 별도로 처리하므로 처리 시간이 오래 걸린다. 또한 독립된 3개의 모델을 모두 사용해야 최종 예측을 할 수 있다.	선택적 탐색 알고리즘의 느린 속도 탓에 여전히 처리 시간이 오래 걸린다.	영역 제안에서 시간 소모가 크다. 또한 여러 시스템이 순차적으로 동작하기 때문에 전체 처리 속도가 각 시스템의 처리 속도에 영향을 많이 받는다.
이미지당 처리 시간	50초	2초	0.2초
R-CNN 대비 처리 속도	1배	25배	250배

정확도* : 2007년 PASCAL 사물 분류 경진대회에서 기록한 mAP

2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN과 그 변종 정리

- **R-CNN**: 경계 박스를 선택적 탐색 알고리즘이 제안한다. 각 경계 박스의 영역 이미지에서 (AlexNet 등의) 합성곱 신경망이 특징을 추출하고, 이 특징을 통해 선형 SVM이 물체의 클래스, 선형 회귀 모델이 경계 박스의 위치를 각각 예측한다.
- **Fast R-CNN**: 완결된 단일 모델이 되도록 R-CNN의 설계를 단순화했다. CNN 뒤에 배치된 RoI 풀링 층이 각 경계 박스 영역의 크기를 고정 크기로 변환하는 것이 특징이다. 모델이 물체의 클래스 레이블과 RoI를 모두 직접 예측한다.
- **Faster R-CNN**: 전체 과정을 하나의 딥러닝 모델로 만든 사물 탐지 모델이다. 선택적 탐색 알고리즘을 영역 제안 신경망으로 대체했는데, 영역 제안 신경망은 CNN 에서 추출한 전체 이미지의 특징 맵으로부터 RoI를 제안하도록 학습된다.

2. 영역 기반 합성곱 신경망(R-CNN)

R-CNN의 단점

- 지금까지 소개한 연구 논문은 모두 R-CNN의 연구 성과를 개선해서 실시간 사물 탐지를 실현하기 위한 노력의 결과물임
- 이들 연구 논문의 성과는 놀라운 수준이지만 실시간 사물 탐지기를 구현하지는 못했음
- R-CNN 및 그 변종에는 다음과 같은 단점이 있음
 - 학습 과정이 번잡하고 시간이 오래 걸린다.
 - 학습이 여러 단계에 걸쳐 일어난다 (예: 영역 제안과 분류).
 - 추론 단계에서 신경망의 속도가 느리다.
- 다행히도 더욱 개선된 새로운 신경망 구조가 발표되면서 R-CNN의 이러한 단점을 극복하고 실시간 사물 탐지를 달성할 수 있었음. 이들 중 가장 널리 알려진 것이 싱글샷 탐지기 single-shot detector, SSD, YOLO you only look once 임

2. 영역 기반 합성곱 신경망(R-CNN)

다단계 탐지기와 단일 단계 탐지기

- R-CNN 및 변종 모델은 모두 영역 기반 방식임
- 탐지 과정이 두 단계로 나뉘므로 이들 탐지기를 2단계 탐지기라고 함
 - 선택적 탐지 알고리즘이나 RPN을 통해 RoI를 추출한다. 가능한 경계 박스 수가 무한대이기 때문에 영역 제안 수는 상대적으로 희소하다.
 - 분류기는 제안된 영역만 처리한다.
- 단일 단계 탐지기(SSD, YOLO)는 이와는 다른 접근 방식을 취함
- 영역 제안 추출 단계를 건너뛰고 이미지 내 가능한 모든 위치를 대상으로 직접 탐지를 시도하는 방식임
- 처리가 빠르고 간단하지만 성능이 소폭 하락하는 경향이 있음
- 대체로 단일 단계 탐지기는 2단계 탐지기에 비해 정확도가 떨어지지만 처리 속도는 훨씬 빠름

3. 싱글샷 탐지기(SSD)

- 싱글샷 탐지기는 2016년 웨이 리우의 논문에서 처음 제안되었음
- PASCAL VOC와 MS COCO 데이터셋을 대상으로 59FPS의 속도와 mAP 74%의 성능을 보이며 속도와 성능 모두 최고 기록을 경신함
- **R-CNN 및 변종**은 다단계 탐지기이며, 경계 박스 내 영역의 물체 존재 확신도를 예측한 다음 이 경계 박스를 분류기로 넘겨 물체의 클래스를 예측하는 방식으로 동작함
- 반면 SSD나 YOLO 같은 단일 단계 탐지기는 **합성곱층에서 위치와 클래스를 한번에** 예측하기 때문에 이름도 그에 걸맞게 싱글샷이라고 명명함
- 물체 존재 확신도는 정답과 경계박스의 중첩률을 계산해서 예측하며, 경계 박스가 정답과 완전히 일치하면 물체 존재 확신도가 1이 되고, 두 경계 박스가 전혀 겹치지 않는다면 0이 됨
- 일반적으로 물체 존재 확신도의 임계값은 0.5로 설정하며, 물체 존재 확신도가 50% 이상이면 해당 영역에는 물체가 있을 확률이 높다고 보고 예측 결과를 채용하지만 50% 이하면 예측 결과를 무시함

3. 싱글샷 탐지기(SSD)

탐지기의 속도 측정(초당 프레임, FPS)

탐지 속도를 측정하는 가장 일반적인 기준은 초당 프레임^{FPS} 이다. 예를 들어 Faster R-CNN의 처리 속도는 7 FPS 이다. 탐지 처리 파이프라인의 각 단계를 개선해서 전체 속도를 향상시키려는 다양한 시도가 있었으나 현재까지는 정확도를 일부 희생하는 방법 외에는 눈에 띄는 속도 개선이 불가능했다. 이 절에서는 SSD와 같은 단일 탐지기가 어떻게 실시간 탐지에 더 적합한 더 빠른 탐지를 달성했는지 살펴볼 것이다.

SSD300은 59FPS의 속도로 mAP 74.3%를 달성했으며, SSD512는 22FPS의 속도로 mAP 76.8%를 달성하며 7FPS의 속도로 mAP 73.2%를 기록한 Faster R-CNN의 성능을 능가했다. SSD300은 입력 이미지 크기가 300x300, SSD512는 512x512 여서 이러한 이름이 붙었다.

3. 싱글샷 탐지기(SSD)

SSD의 추상적 구조

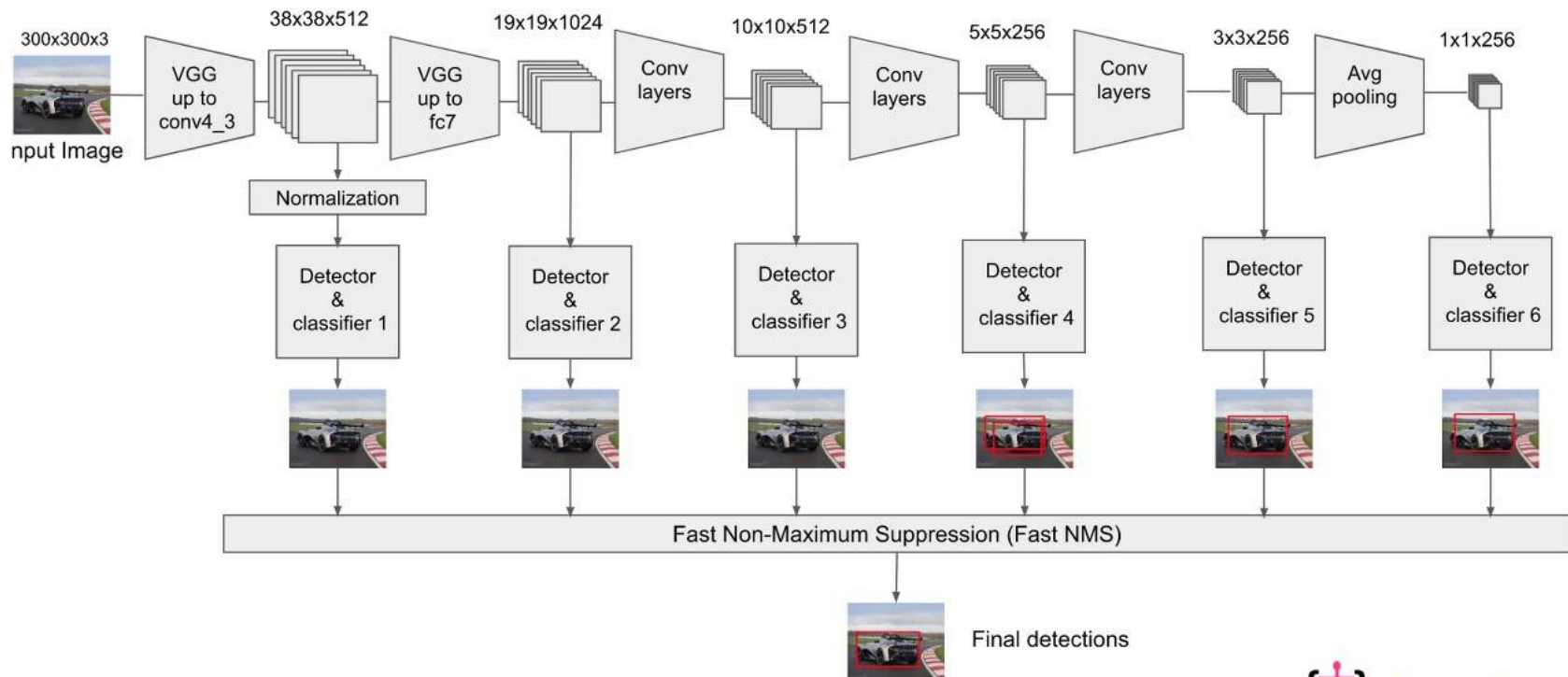
- SSD는 피드포워드^{feedforward} 합성곱 신경망 구조를 가지며, 여러 개의 고정 크기 경계 박스^{anchor box}를 생성하고 각 박스에 클래스별 물체 존재 확신도를 부여한 다음 비최대 억제 알고리즘^{non maximum suppression, NMS}을 통해 최적 탐지 결과를 제외한 나머지를 배제하는 파이프라인을 가짐
- SSD 모델은 크게 다음 세 가지 요소로 구성됨
 - **특징 맵을 추출하는 기본 신경망**: 고해상도 이미지 분류에 사용되는 사전 학습된 신경망에서 분류기 부분을 제거한 것이다. 논문에서는 VGG16을 사용했다. VGG16 혹은 ResNet 등을 사용해도 마찬가지로 좋은 결과를 얻을 수 있다.
 - **다중 스케일 특징층**: 기본 신경망 뒤에 배치된 일련의 합성곱 필터다. 이들 층은 점진적으로 필터 크기가 감소하며 둘 이상의 배율로 탐지를 시도한다.
 - **비최대 억제**: NMS를 적용해서 중복되는 경계 박스를 배제하고 물체별로 경계 박스를 하나만 남긴다.

3. 싱글샷 탐지기(SSD)

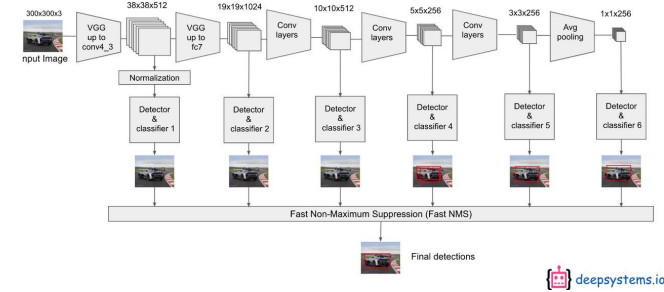
■ 3.1 SSD의 추상적 구조

- SSD 모델은 총 4_3, 7, 8_2, 9_2, 10_2, 11_2 가 NMS로 직접 예측 결과를 전달함
- 이들 층의 필터 크기는 점진적으로 감소하는 형태를 취함 (38→19→10→5→3→1)

SSD 모델 기본 신경망(VGG16)



3. 싱글샷 탐지기(SSD)



■ 3.1 SSD의 추상적 구조

- 클래스당 8732건이 탐지되며 이를 NMS층 (비최대 억제층)에 전달해서 물체 하나에 탐지 한 건만 남기는 것을 알 수 있음

- 8732건 숫자는 어디서 온 것인가?

- CONV4_3: $38 \times 38 \times 4 = 5776$ (각 위치마다 4개 박스)
- CONV7: $19 \times 19 \times 6 = 2166$ (각 위치마다 6개 박스)
- CONV8_2: $10 \times 10 \times 6 = 600$ (각 위치마다 6개 박스)
- CONV9_2: $5 \times 5 \times 6 = 150$ (각 위치마다 6개 박스)
- CONV10_2: $3 \times 3 \times 4 = 36$ (각 위치마다 4개 박스)
- CONV11_2: $1 \times 1 \times 4 = 4$ (각 위치마다 4개 박스)
- 전체: $5776 + 2166 + 600 + 150 + 36 + 4 = 8732$

위치마다의 박스 수는
파라미터 인가?

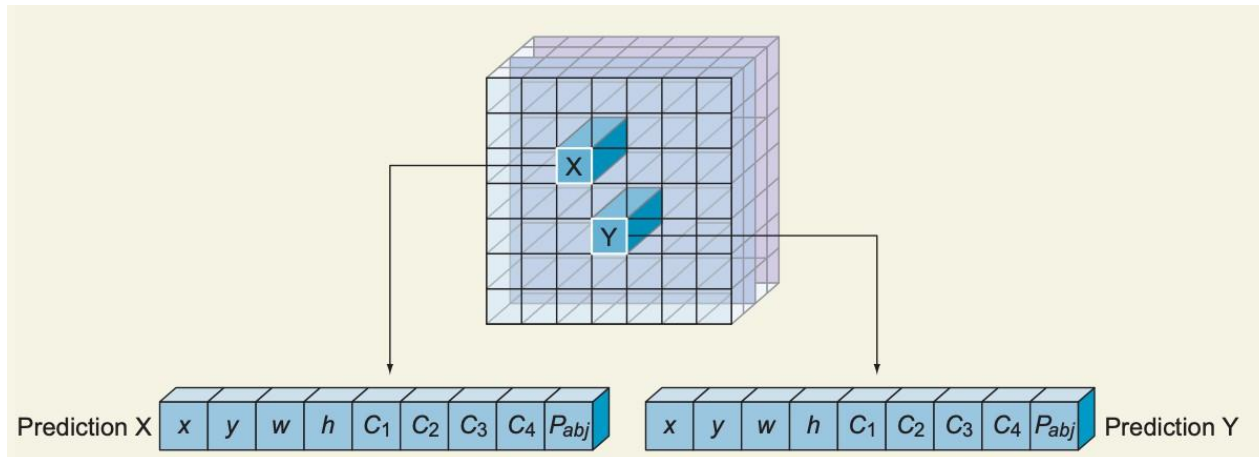
- 이들 경계박스를 그대로 출력하기엔 경계 박스 수가 너무 많아 NMS를 적용해서 출력되는 경계 박스 수를 줄임 (cf. YOLO는 98개)

3. 싱글샷 탐지기(SSD)

예측 결과는 어떤 모양일까?

- 신경망은 각 특징에 대해 다음과 같은 값을 예측함
 - 경계 박스를 결정하는 값 (4개): (x, y, w, h)
 - 물체 존재 확신도 (1개)
 - 각 클래스의 확률을 나타내는 값 (C 개)
- 예측 결과는 $5+C$ 개 값으로 구성됨

분류 대상 클래스 수가 4개인 문제에 대한 예측 결과 시각화



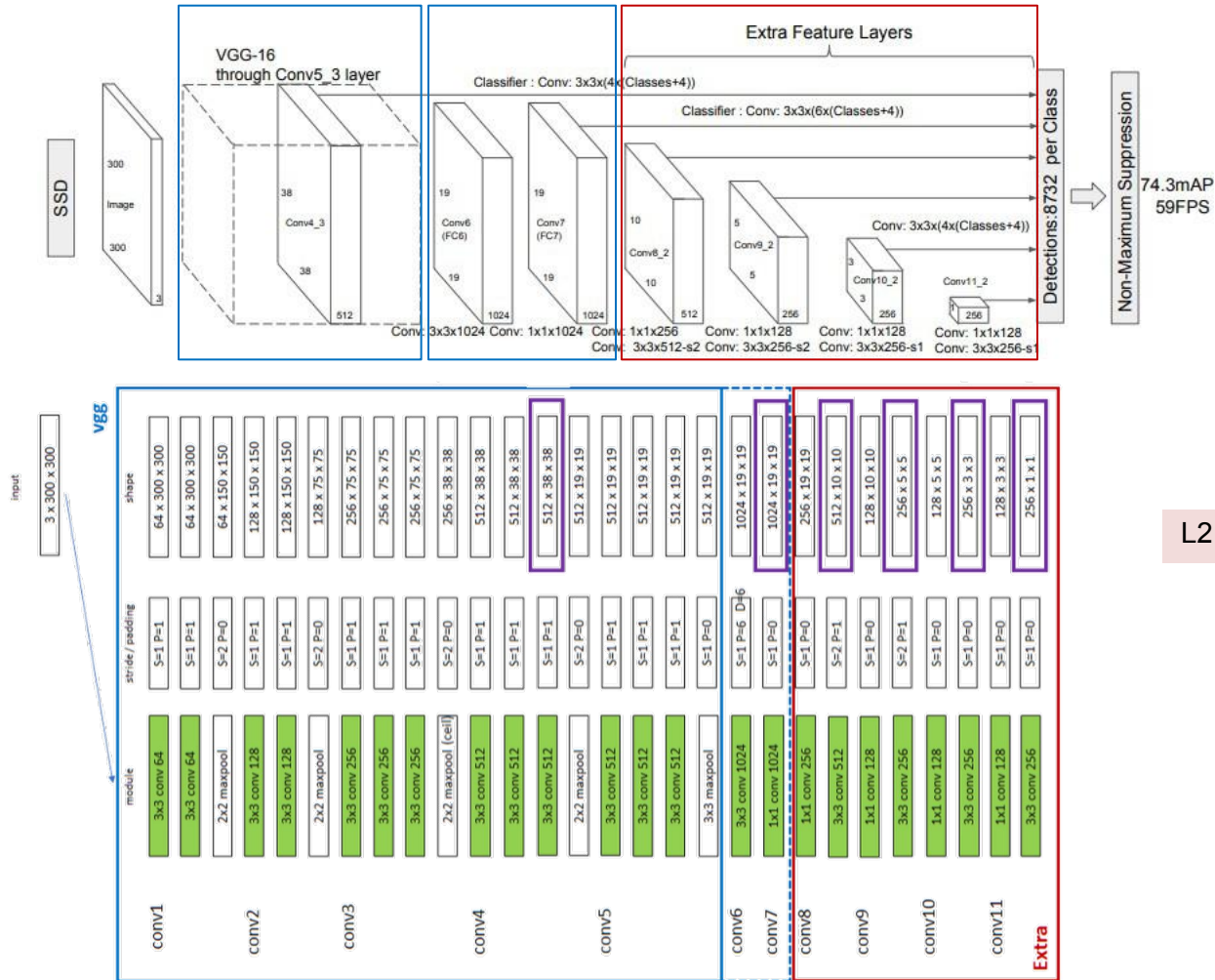
3. 싱글샷 탐지기(SSD)

기본 신경망

- SSD 신경망 구조에는 VGG16 신경망을 기본 신경망으로 사용하였음
- 분류기에 해당하는 전결합층을 제거하고, 보조 합성곱층을 추가해 사용함
- VGG16 신경망이 기본 신경망으로 채택된 이유는 고해상도 이미지 분류에서 높은 성능을 보였으며, 다양한 문제에서 전이학습으로 좋은 성과를 낸 사례가 많기 때문임
- VGG16을 VGG19나 ResNet으로 대체하더라도 늘어난 층수로 인해 속도나 정확도가 하락할 수는 있으나 동작에는 문제 없음

3. 싱글샷 탐지기(SSD)

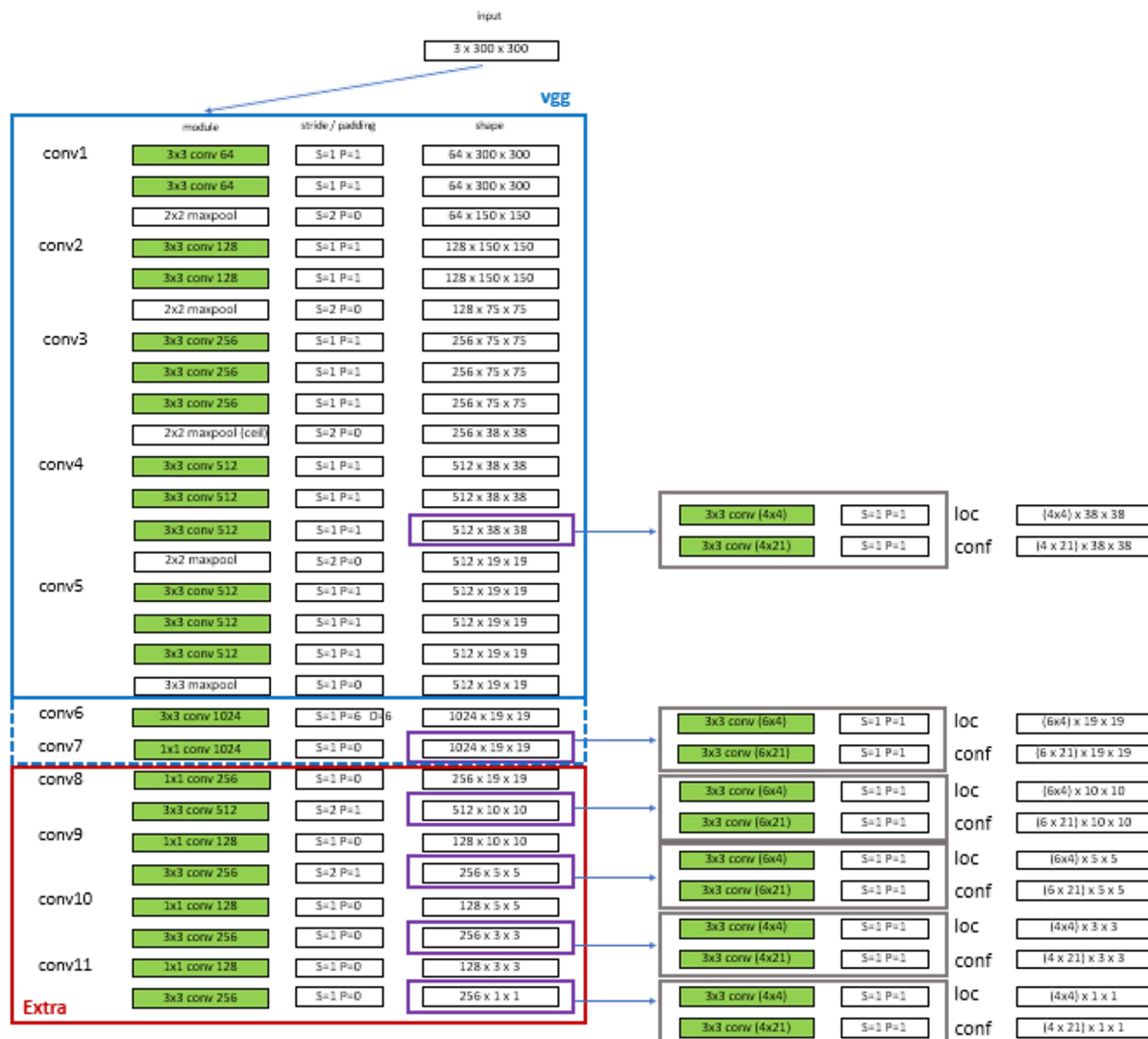
기본 신경망



L2 normalization technique

3. 싱글샷 탐지기(SSD)

기본 신경망



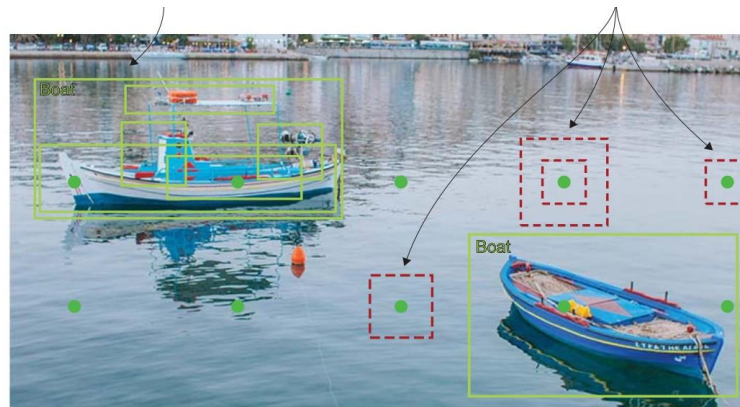
3. 싱글샷 탐지기(SSD)

기본 신경망의 예측 과정

- 예제를 통해 기본 신경망의 예측 과정을 살펴보면 다음과 같음
 - 1) SSD에는 R-CNN의 앵커 박스와 비슷한 역할을 하는 격자가 있다. 각 격자점마다 해당 앵커를 중심으로 하는 경계 박스를 생성한다. SSD에서는 이들 격자점을 프라이어^{prior}라고 한다.
 - 2) SSD에서는 각 경계 박스를 별도의 이미지로 간주한다. 경계 박스마다 ‘이 경계 박스 안에 보트가 있는가?’, 좀 더 정확히 말하자면 ‘이 경계 박스 안에서 보트의 특징을 뽑았는가?’를 확인한다.
 - 3) 보트의 특징을 포함하는 경계 박스를 발견했다면 해당 박스의 예측 좌표와 사물 분류 결과를 NMS 층으로 전달한다.
 - 4) NMS 층은 정답과 중첩률이 가장 높은 경계 박스 하나만 남기고 다른 경계 박스는 배제한다.

보트의 특징을 발견한 경계 박스

보트의 특징을 발견하지 못한 경계 박스



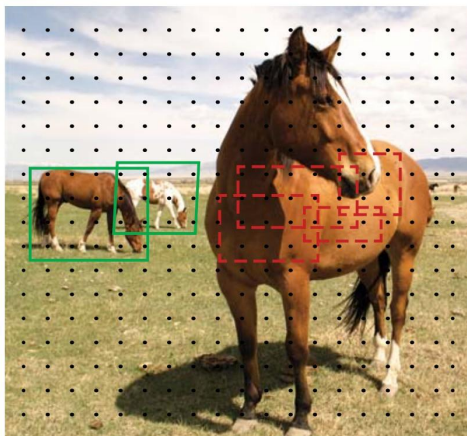
3. 싱글샷 탐지기(SSD)

■ 다중 스케일 특징층

- 다중 스케일 특징층은 기본 신경망 뒤로 이어지는 일련의 합성곱층임
- 합성곱 필터의 크기가 점진적으로 감소하도록 배치되어 다양한 배율로 예측 및 탐지를 수행할 수 있는 것이 특징임

■ 다중 스케일 탐지

- 앵커 박스의 크기를 벗어나는 물체는 검출하기 어려움

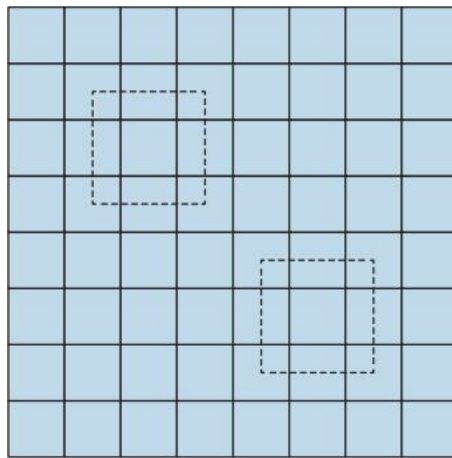


카메라에서 멀리 떨어진 말은 앵커 박스 안에 포함될 가능성이 높아서 탐지하기 쉽다. 반면 카메라와 가까운 말은 기본 신경망에서 탐지되지 않을 가능성이 높다. 이를 탐지하려면 큰 범위의 특징을 포착할 수 있도록 다른 스케일의 앵커 박스가 필요하다.

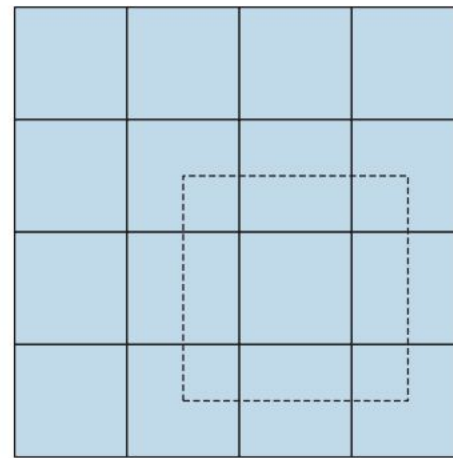
3. 싱글샷 탐지기(SSD)

다중 스케일 탐지

- 이미지에 서로 다른 비율로 들어 있는 물체를 처리하려면 이미지를 서로 다른 크기로 전처리하고 그 결과를 합쳐서 처리할 방법이 필요
- 합성곱 필터이 크기를 달리한 여러 개의 합성곱층을 사용하면 한 신경망 안에서도 이와 같은 효과를 얻을 수 있으며 여러 비율에 해당하는 파라미터를 전체 신경망에서 공유할 수 있음



8 × 8 feature map



4 × 4 feature map

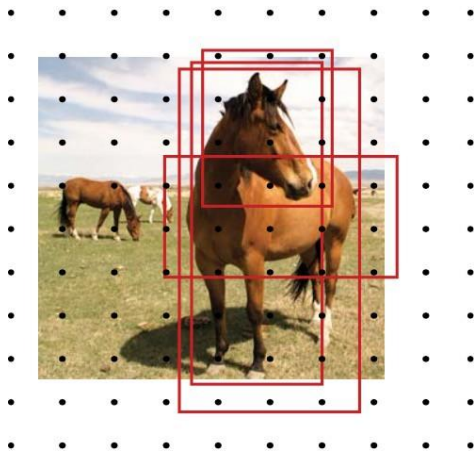
고해상도 특징맵 (작은물체용)

저해상도 특징맵 (큰물체용)

3. 싱글샷 탐지기(SSD)

다중 스케일 탐지

- CNN은 점진적으로 이미지의 크기를 줄이므로 특징 맵의 크기도 함께 감소함
- 이렇게 줄어든 특징 맵을 큰 크기의 물체를 탐지하는데 활용



다중 스케일 특징층은 입력 이미지의 크기를 축소해서 서로 다른 크기로 나타난 대상을 탐지할 수 있다.

3. 싱글샷 탐지기(SSD)

다중 스케일 탐지

- 다중 스케일 특징 맵 사용을 통해 신경망의 정확도가 크게 개선되었음
- 여러 층에서 서로 다른 배율로 경계 박스를 만드는 것이 성능 개선에 효과적
 - 출력을 직접 전달하는 다중 스케일 특징층 수가 6개인 경우와 하나인 경우의 정확도가 각각 74.3%와 62.4%로 큰 차이를 보였음

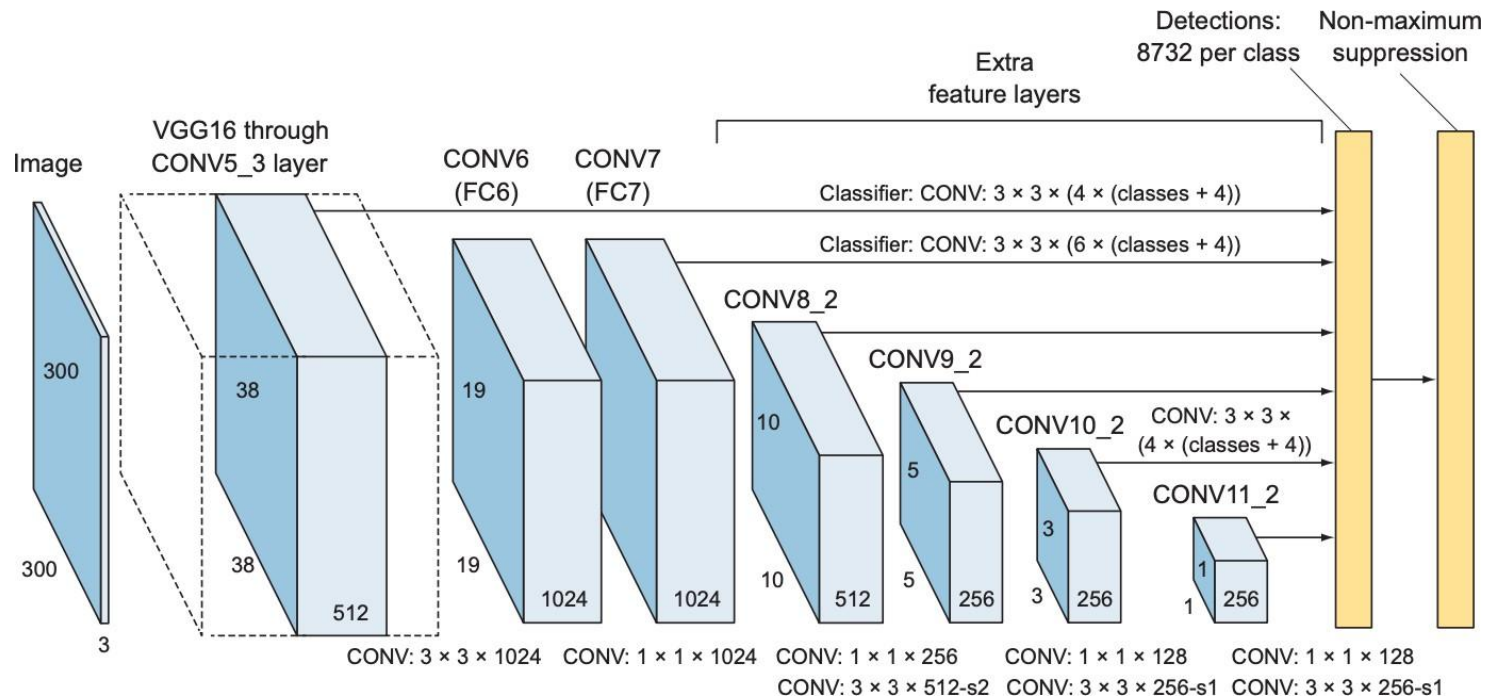
다중 스케일 특징 맵의 효과를 확인한 실험 결과 (다중 스케일 특징 맵을 반영하니 정확도가 증가함)

출력층에 출력을 직접 전달한 층						다중 스케일 특징 맵 사용 여부		경계 박스 수
conv4_3	conv7	conv8_2	conv9_2	conv10_2	conv11_2	Yes	No	
✓	✓	✓	✓	✓	✓	74.3	63.4	8,732
✓	✓	✓	✓	✓		74.6	63.1	8,764
✓	✓	✓	✓			73.8	68.4	8,942
✓	✓	✓				70.7	69.2	9,864
✓	✓					64.2	64.4	9,025
	✓					62.4	64.0	8,664

3. 싱글샷 탐지기(SSD)

다중 스케일 특징층의 구조

- SSD 연구진은 크기가 점차 감소하는 여섯 층의 합성곱층을 추가 했는데, 이 층수를 결정하는 과정에서 많은 시행착오를 거쳤음
 - CONV6은 커널크기가 3x3, CONV7은 커널크기가 1x1로 서로 독립된 층
 - 8번째부터 11번째 층은 커널 크기가 각각 1x1, 3x3인 2개의 합성곱층이 합쳐져 구성된 블록으로 이루어져 있음



3. 싱글샷 탐지기(SSD)

다중 스케일 특징층의 구조

- 학계나 연구에 종사하는 사람이 아니라면 사물 탐지 시스템을 직접 구현할 필요는 없음
- 대개는 오픈 소스로 구현된 소프트웨어를 내려 받아 자신의 필요에 맞게 사용할 뿐임
- 다음은 6번째 층부터 11번째 층을 케라스로 구현한 코드임
 - 이 코드 조각을 책에 실은 이유는 구조를 이해하는데 도움을 주기 위함

```
# conv6 and conv7
conv6 = Conv2D(1024, (3, 3), dilation_rate=(6, 6), activation='relu',
padding='same')(pool5)
conv7 = Conv2D(1024, (1, 1), activation='relu', padding='same')(conv6)

# conv8 block
conv8_1 = Conv2D(256, (1, 1), activation='relu', padding='same')(conv7)
conv8_2 = Conv2D(512, (3, 3), strides=(2, 2), activation='relu',
padding='valid')(conv8_1)

# conv9 block
conv9_1 = Conv2D(128, (1, 1), activation='relu', padding='same')(conv8_2)
conv9_2 = Conv2D(256, (3, 3), strides=(2, 2), activation='relu',
padding='valid')(conv9_1)

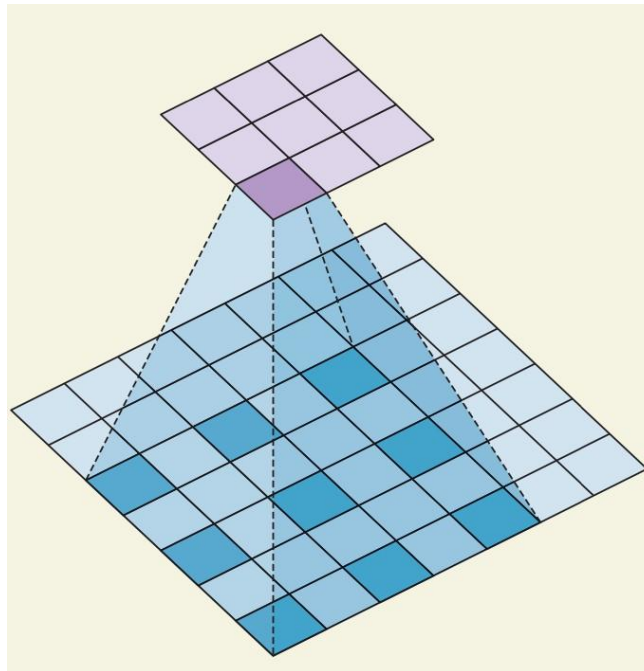
# conv10 block
conv10_1 = Conv2D(128, (1, 1), activation='relu', padding='same')(conv9_2)
conv10_2 = Conv2D(256, (3, 3), strides=(1, 1), activation='relu',
padding='valid')(conv10_1)

# conv11 block
conv11_1 = Conv2D(128, (1, 1), activation='relu', padding='same')(conv10_2)
conv11_2 = Conv2D(256, (3, 3), strides=(1, 1), activation='relu',
padding='valid')(conv11_1)
```


3. 싱글샷 탐지기(SSD)

팽창 합성곱

- 팽창 합성곱 ^{dilated convolution}에는 새로운 파라미터인 팽창률 ^{dilated rate}이 도입됨
- 팽창률이란 커널과 대상 이미지의 거리를 의미함
- 팽창률 2를 적용한 3x3커널은 5x5커널에서 9개 파라미터만 사용하는 것과 동등함
- 행과 열을 하나씩 건너 제거한 5x5 커널을 상상하면 이해하기 쉬움



3. 싱글샷 탐지기(SSD)

7.3.3.3 팽창 합성곱

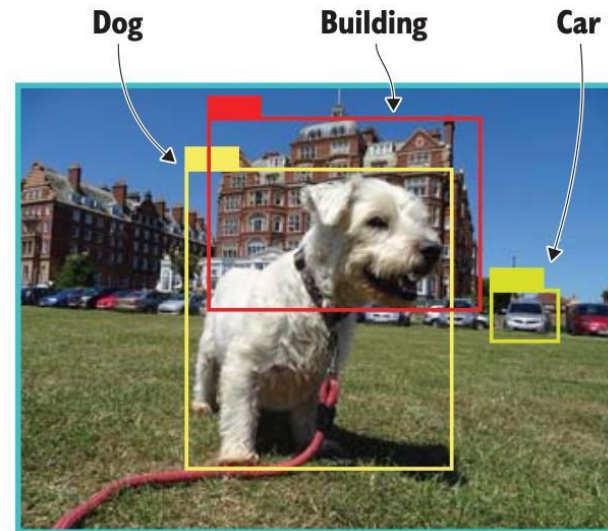
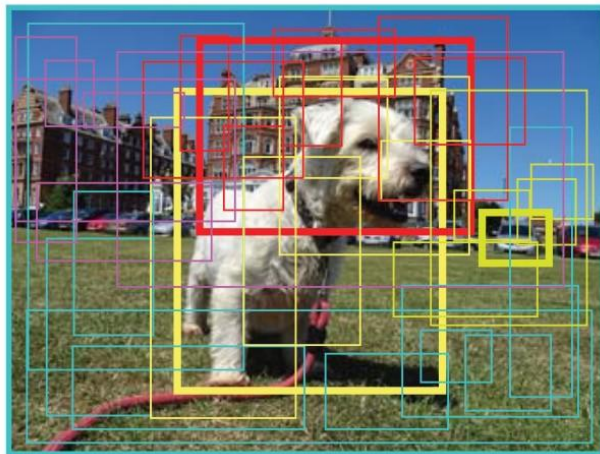
- 팽창 합성곱 ^{dilated convolution}에는 새로운 파라미터인 팽창률 ^{dilated rate}이 도입됨
- 팽창률이란 커널과 대상 이미지의 거리를 의미함
- 팽창률 2를 적용한 3x3커널은 5x5커널에서 9개 파라미터만 사용하는 것과 동등함
- 행과 열을 하나씩 건너 제거한 5x5 커널을 상상하면 이해하기 쉬움
- 팽창 합성곱은 같은 계산 비용으로 더 넓은 감수 영역의 효과를 볼 수 있음
- 팽창 합성곱은 특히 실시간 세그멘테이션 분야에서 널리 사용됨
- 합성곱 연산을 여러 번 적용하거나 큰 커널을 적용할 계산 자원의 여유가 없는 상황에서
넓은 범위의 감수 영역이 필요할 때 사용하면 좋음
- 다음은 팽창률 2를 적용한 3x3 팽창 합성곱층을 케라스로 구현한 코드임

```
Conv2D(1024, (3, 3), dilation_rate=(2,2), activation='relu', padding='same')
```

3. 싱글샷 탐지기(SSD)

비최대 억제

- 추론 시점에 SSD의 순방향 계산에서 생성되는 많은 수의 경계 박스는 대부분 NMS 알고리즘을 통해 배제됨
- 확신도 손실이 부여되었거나 중첩률이 임계값보다 낮은 경계 박스는 폐기되며, 상위 N개의 예측만 그대로 남음
- 이러한 과정을 통해 가장 가능성이 높은 예측만 결과로 출력됨



3. 싱글샷 탐지기(SSD)

비최대 억제

- NMS는 어떤 방식으로 경계 박스를 배제하는 것인가?
 - 먼저 확신도를 기준으로 경계 박스를 정렬
 - 확신도가 가장 높은 경계 박스부터 그 이전에 본 각각의 경계 박스와 중첩률을 계산한 다음
그중 현재 경계 박스와 같은 클래스로 분류되었고 일정 중첩률을 초과하는 것이었는지 확인
 - 임계값보다 중첩률이 높은 경계 박스는 확신도가 더 높은 경계 박스와 지나치게 겹치므로 같은 물체를 가리킨다고 판단해서 폐기됨
 - 이런 식으로 이미지 한 장당 최대 200개의 경계 박스를 남김

4. YOLO

- YOLO는 2016년 조셉 레드몬이 최초로 제안한 사물 탐지 신경망
- R-CNN과 비슷하게 수년에 걸쳐 다음과 같이 개선된 파생 버전이 제안되었음
 - YOLOv1(2016) : 사물 탐지와 클래스 분류를 하나로 통합한 설계를 적용했기 때문에 통합 실시간 사물 탐지라고 불렀다.
 - YOLO Evolution(<https://arxiv.org/pdf/2411.00201>)

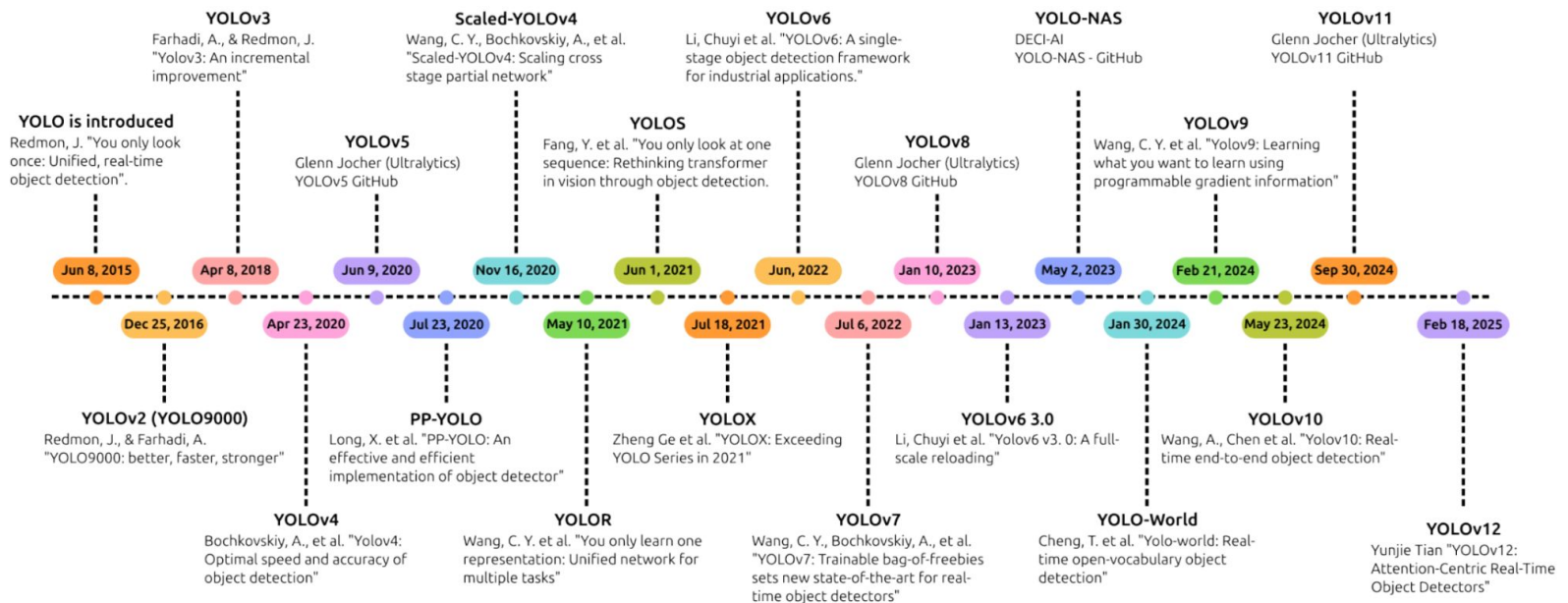


Figure 1: Evolution of YOLO Algorithms throughout the years

4. YOLO

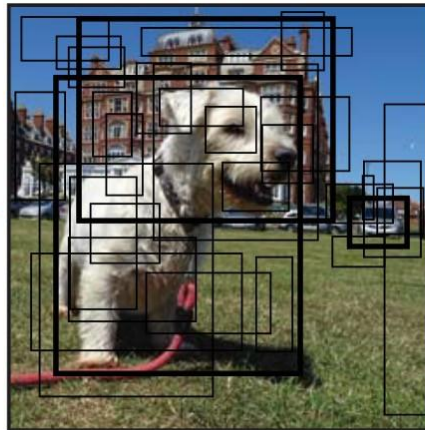
- YOLO 및 그 변종은 모두 전체 처리 과정이 딥러닝으로 구현된 고속 사물 탐지 모델이며, 고속 실시간 사물 탐지에 도전한 초기 시도 중 한가지 임
- 현재도 사물 탐지 알고리즘 중에서 속도가 빠른 편에 속함
- 정확도는 R-CNN에 미치지 못하지만 빠른 처리 속도 덕분에 사물 탐지 분야에서는 인지도가 높으며 **실시간 영상 또는 카메라 입력 처리 데모용**으로 자주 사용됨
- YOLO는 지금까지 설명했던 신경망과는 조금 다른 접근법을 취함
- R-CNN과 같은 영역 제안 단계가 아예 없음
- 그 대신 입력을 비교적 소수의 격자 형태로 분할하고 분할된 영역을 대상으로 직접 경계 박스와 사물 분류를 수행함
- 그 결과 많은 수의 경계 박스 후보가 생성되는데 이를 다시 NMS를 사용해서 최종 예측 결과로 좁히는 방식임

4. YOLO

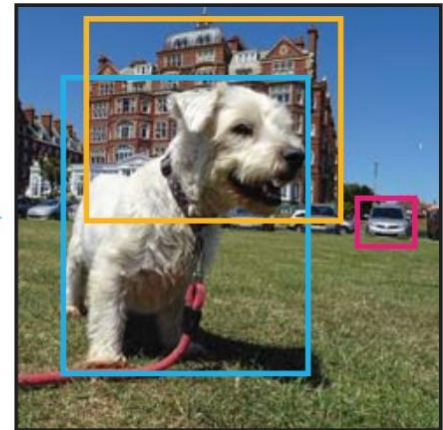
- YOLOv1 에서는 일반적인 구조가 제안되었고, YOLOv2 에서는 설계가 다듬어지고 경계 박스 제안에 사전 정의된 앵커 박스가 도입되었음
- YOLOv3에서는 모델 구조 및 학습 과정이 더욱 더 보완되었음
- 이 절에서는 YOLO 변종 중 가장 높은 성능 기록을 가진 YOLOv3를 중심으로 YOLO 처리 과정을 살펴보겠음



이미지를 격자 형태로 분할



경계 박스를 예측하고
물체 클래스를 분류



NMS를 사용해 결과
후보의 범위를 좁힘

4. YOLO

YOLOv3 처리 과정

- YOLO 처리 과정 중 첫 번째 단계는 이미지를 $S \times S$ 개의 격자 모양으로 분할하는 것
- 정답 경계 박스의 중심이 이 중 어떤 조각에 포함된다면 해당 조각에서 물체의 존재가 탐지되어야 함
- 다음과 같은 방법으로 분할된 각 조각마다 B개의 경계 박스와 물체 존재 확신도, 물체의 클래스를 예측함
 - B개의 경계 박스를 결정하는 좌표: 앞서 살펴본 사물 탐지 알고리즘과 마찬가지로 YOLO 역시 경계박스를 4개의 값을 가진 튜플로 정의함. 이때 x 와 y 는 조각 위치에 대한 오프셋임
 - 물체 존재 확신도(P_0): 해당 이미지 조각에 물체가 포함되어 있을 확률. 물체 존재 확신도는 시그모이드 함수를 거쳐 0과 1사이의 값으로 변환되어 확률로 다뤄짐. 물체 존재 확신도는 다음과 같이 계산함
 - $P_0 = P_r(\text{물체포함}) \times \text{중첩률(예측, 정답)}$
 - 물체의 클래스 분류: 경계 박스에 물체가 포함되었다면 K개 클래스에 대해 물체가 해당 클래스에 속할 확률을 계산함

4. YOLO

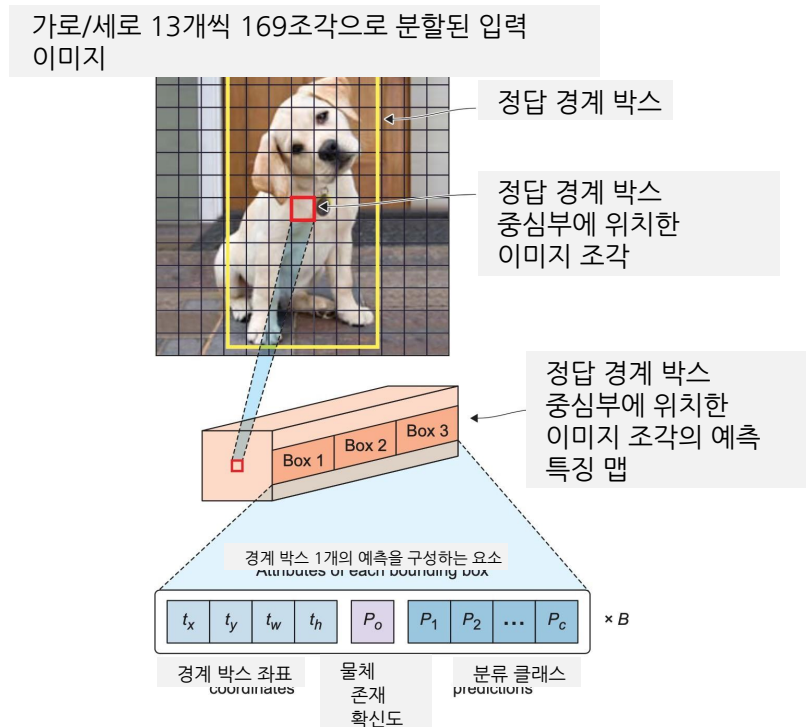
YOLOv3 처리 과정

- YOLOv1-2 에서는 클래스 분류 확률을 구하기 위해 소프트맥스 함수를 사용함
- YOLOv3 에서는 소프트맥스 대신 시그모이드 함수를 사용함
- 함수를 변경한 이유는 소프트맥스 함수를 사용하려면 경계 박스 하나가 항상 하나의 클래스에 만 속한다는 가정이 필요하기 때문
- 즉, 어떤 물체가 반드시 한 가지 클래스에만 속하지는 않고 <사람과 여자>처럼 한 물체가 둘 이상의 클래스를 갖는 데이터셋도 존재함
- 이와 같이 다중 레이블을 부여할 수 있는 모델은 정확도가 좀 더 높음

4. YOLO

YOLOv3 처리 과정

- 하나의 경계 박스(B)마다 [(경계 박스 좌표), (물체 존재 확신도), (분류 클래스)] 형태의 예측 결과가 부여되어 있음
 - 경계 박스 좌표 4개, 물체 존재 확신도 1개, K개 클래스 각각에 대한 확률
- 예측된 값의 개수 $\rightarrow B * (4 + 1 + K) * (S * S)$

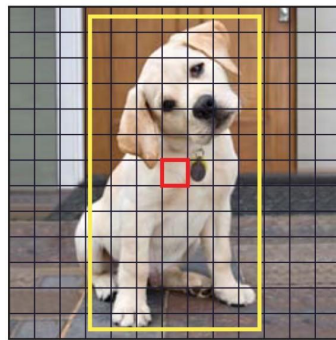


4. YOLO

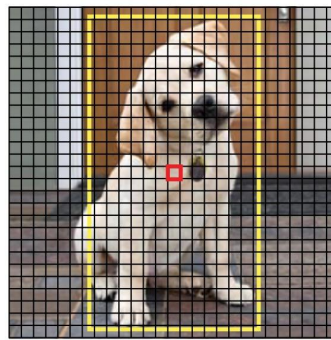
YOLOv3 처리 과정

- 서로 다른 크기의 세 가지 특징 맵에서 각각 경계 박스를 하나씩 예측하므로 결과적으로 이미지 조각 하나당 3개의 경계 박스 Bounding Box가 예측 결과로 나옴
- 개를 탐지할 수 있는 경계 박스는 앵커가 정답 경계 박스와 함께 가장 높은 **중첩률** IoU 을 가지는 박스가 될 것임
- <다양한 배율로 탐지를 시도하는 전략>은 YOLOv2의 단점이었던 작은 물체를 탐지하지 못하는 문제를 해결하는데 도움이 되었으며, <업샘플링층>은 신경망이 작은 물체를 탐지하는데 중요한 정보가 되는 세밀한 특징을 보존 할 수 있게 해주었음

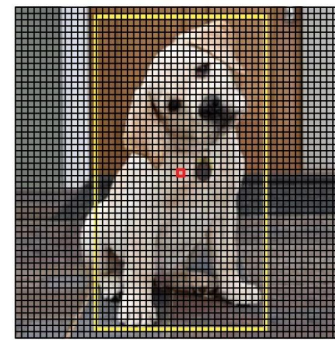
서로 다른 배율의 예측 특징 맵



13 × 13



26 × 26



52 × 52

4. YOLO

YOLOv3 가 출력하는 경계 박스

입력 이미지의 크기가 416x416 일 때 YOLO는

$((52 \times 52) + (26 \times 26) + (13 \times 13)) \times 3 = 10,647$ 개의 경계 박스를 예측 결과로 내놓음

10,647이라는 개수는 우리가 활용하기에 너무 많은 숫자이기에 경계 박스 예측을 줄이는 것이 필요

경계 박스 예측 수를 줄이는 방법

첫번째는 물체 존재 확신도가 미리 정한 임계값을 넘는지를 기준으로 걸러내는 방법

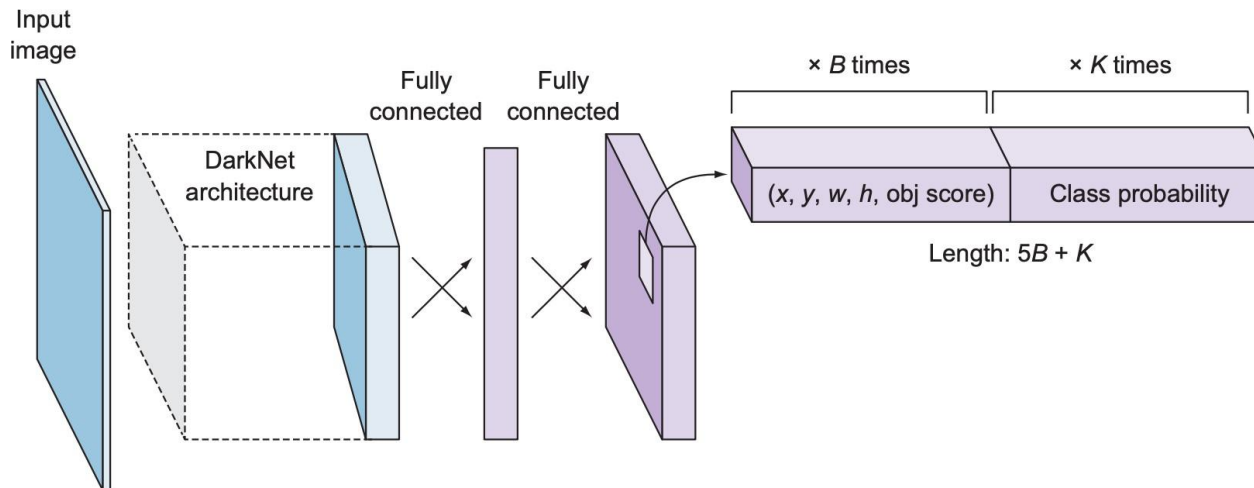
두번째는 NMS를 사용해서 같은 물체를 가리키는 여러 개의 경계 박스를 제거하는 방법

- 즉, 격자로 나눈 입력 이미지의 한 조각에서 예측한 3개의 경계 박스가 모두 하나의 상자를 탐지했는데, 인접한 조각에서도 같은 물체를 탐지했을 수도 있음

4. YOLO

YOLOv3 구조

- YOLO는 사물 탐지와 클래스 분류를 모두 수행하는 단일 신경망임
- 신경망의 구조는 GoogLeNet(인셉션)의 특징 추출기 부분을 참고했음
- 다만 인셉션 모듈 대신 1×1 축소층과 3×3 합성곱층을 조합한 구조를 사용함
- 레드몬과 파라디는 이 구조를 다크넷^{DarkNet} 이라고 이름 붙였음



4. YOLO

YOLOv2 구조

- YOLOv2에는 darknet-19라는 구조가 사용되었음
- 이 구조에 사물 탐지를 보조하는 11개의 층을 더해 30층 구조가 되었음
- 30층이나 되는 층수 때문에 작은 물체를 탐지하는데 어려움을 많이 겪었음
- 입력이 층을 거치며 여러 번 다운 샘플링 되면서 세세한 특징을 잃어버리기 때문
- YOLOv2에는 현재 최고 성능을 보이는 알고리즘에 기본적으로 포함되는 주요 요소인 잔차 블록, 스킵 연결, 업샘플링 등이 적용되지 않았음

Convolutional 의 수19개

Darknet-19 특징 추출기 구조

Type	Filters	Size/Stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

4. YOLO

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	171
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

YOLOv3 구조

- YOLOv3는 다크넷 구조의 개선 버전인 darknet-53을 채용함
- darknet-53은 이미지넷 데이터셋을 학습한 53층 구조의 신경망임
- YOLOv3 신경망은 darknet-53에 사물 탐지를 보조하는 53개의 층을 덧붙여, 106층에 달하는 전결합 합성곱 구조가 만들어졌음
- YOLOv3가 YOLOv2에 비해 처리 속도가 오래 걸리는 이유는 여기에 있으나 속도를 희생한 이상으로 정확도에서는 큰 폭의 상승이 있었음

Convolutional 의 수52개 + Connected 의 수 1개

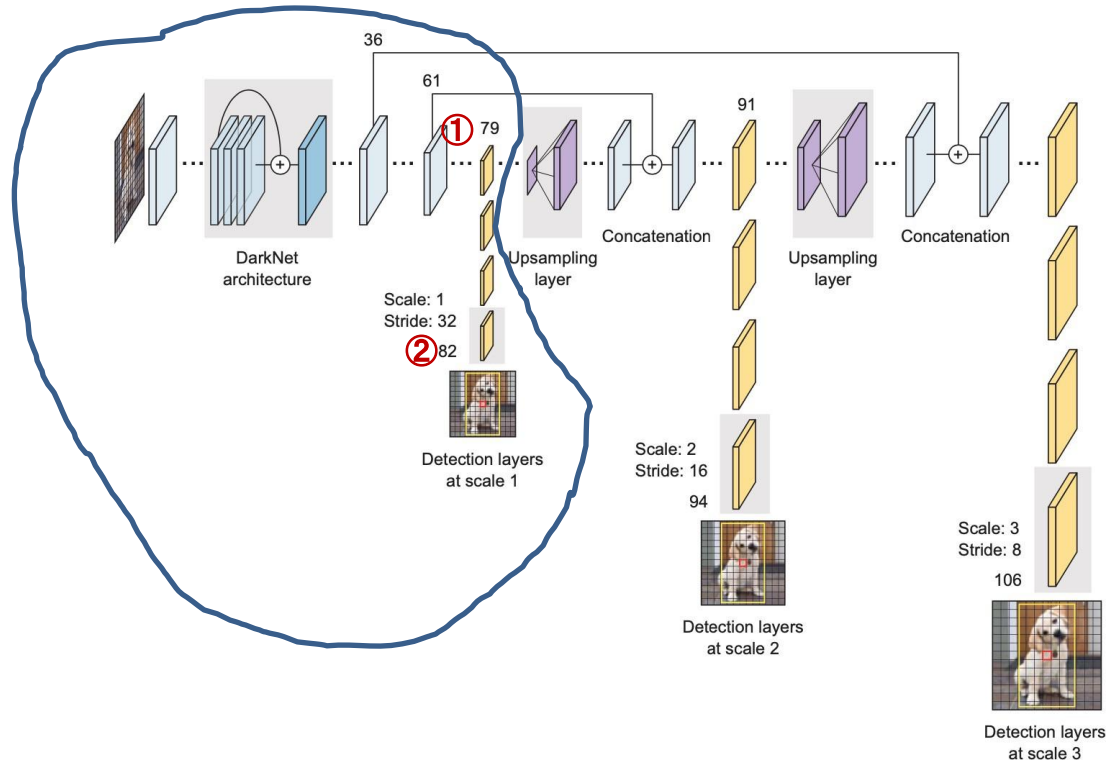
Darknet-53 특징 추출기 구조

	Type	Filters	Size	Output
1×	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
	Convolutional	32	1 × 1	128 × 128
	Convolutional	34	3 × 3	
	Residual			
2×	Convolutional	128	3 × 3 / 2	64 × 64
	Convolutional	64	1 × 1	64 × 64
	Convolutional	128	3 × 3	
	Residual			
	Convolutional	256	3 × 3 / 2	32 × 32
8×	Convolutional	128	1 × 1	32 × 32
	Convolutional	256	3 × 3	
	Residual			
	Convolutional	512	3 × 3 / 2	16 × 16
	Convolutional	256	1 × 1	16 × 16
8×	Convolutional	512	3 × 3	
	Residual			
	Convolutional	1024	3 × 3 / 2	8 × 8
	Convolutional	512	1 × 1	8 × 8
	Convolutional	1024	3 × 3	
4×	Residual			
	Avgpool		Global	
	Connected		1000	
	Softmax			

4. YOLO

YOLOv3 전체 구조

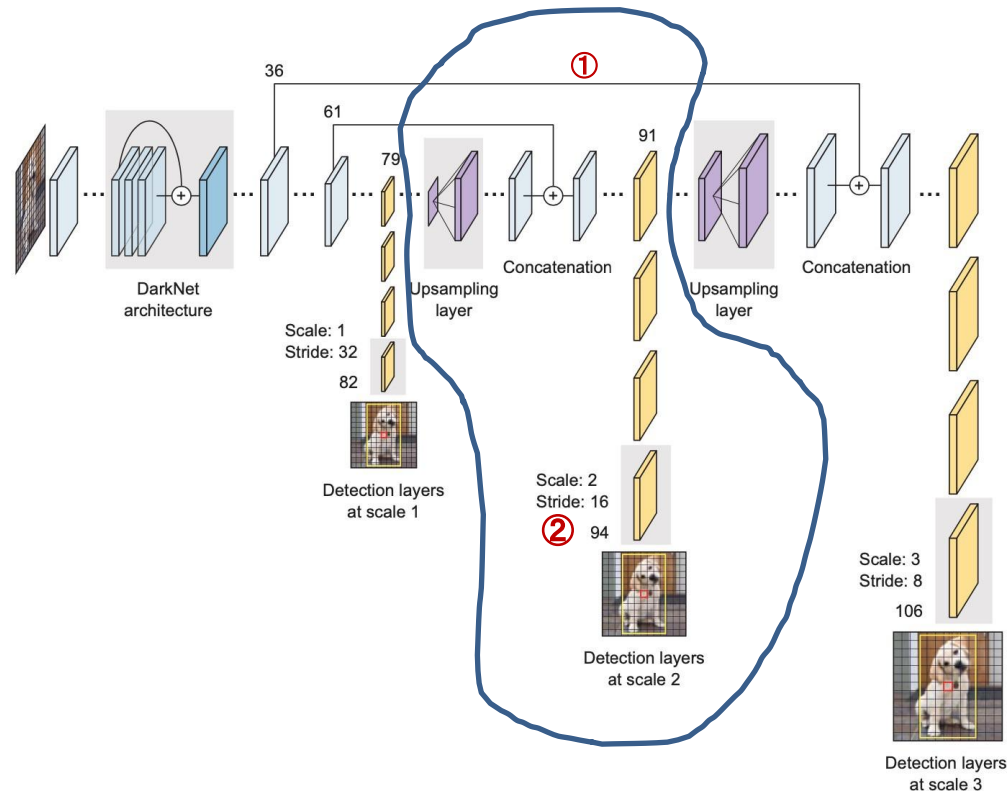
- YOLOv3는 세 가지 다른 배율로 사물 탐지를 수행함
- 입력이미지는 darknet-53 특징 추출기를 지나며 ① 79번 층에 이를 때까지 다운 샘플링됨
- 이 부분에서 신경망이 분기해 첫 번째 탐지가 수행되는 ② 82번 층까지 다운 샘플링이 이어짐
- 이 첫 번째 탐지는 앞서 설명했듯이 13x13 격자로 이미지를 분할하며 큰 물체의 탐지를 담당함



4. YOLO

YOLOv3 전체 구조

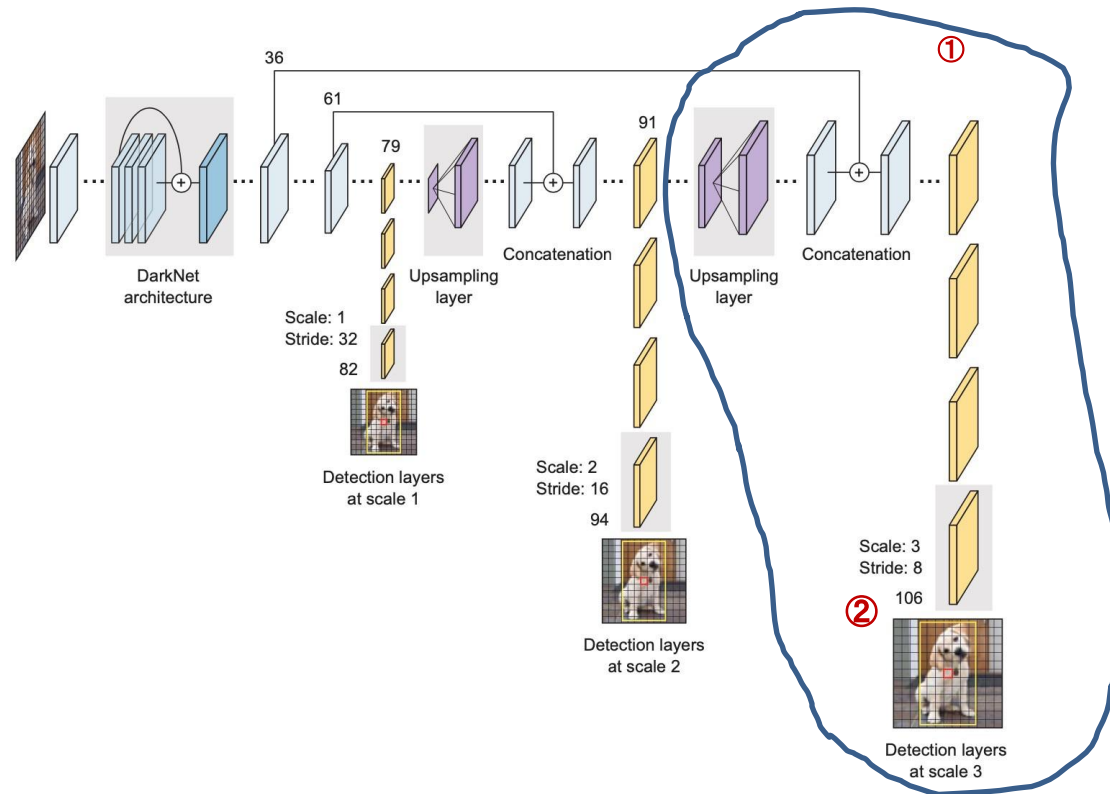
- 다른 쪽 분기는 79번 층의 특징 맵이 두 배로 업샘플링되어 26x26 격자로 분할된 이미지 조각을 형성한 후
① 61번 층의 특징 맵과 연결됨
- 이 부분에서 신경망이 분기해 두 번째 탐지가 수행되는 ② 94번 층까지 다운 샘플링이 이어짐
- 26x26 격자로 분할된 이미지에서 중간 크기 물체의 탐지를 담당하는 두 번째 탐지가 수행됨



4. YOLO

YOLOv3 전체 구조

- 마지막으로 비슷한 과정이 한 번 더 일어남
- 91번 층의 특징 맵이 몇 차례 업샘플링을 거친 후 ① 36번 층의 특징 맵을 깊이 방향으로 연접함
- 그리고 ② 106번 층에서 52x52 격자로 분할된 이미지를 대상으로 작은 크기 물체의 탐지를 담당하는 세 번째 탐지가 수행됨



요약

- 이미지 분류는 이미지에 있는 하나의 물체 종류 또는 클래스를 예측하는 과업이다.
- 사물 탐지는 이미지 속 물체의 위치를 경계 박스를 통해 예측하고 그와 더불어 해당 물체의 클래스를 예측하는 과업이다.
- 일반적인 사물 탐지 프레임워크는 영역 제안, 특징 추출 및 예측, 비최대 억제, 평가 지표 등 네 가지 구성 요소를 갖는다.
- 사물 탐지 알고리즘의 성능을 가늠하는 지표는 크게 속도와 정확도 두 가지다.
속도는 초당 프레임^{FPS}으로 측정하고, 평균평균정밀도^{mAP}로 정확도를 측정한다.
- 사물 탐지 시스템 중 가장 널리 알려진 것은 R-CNN 및 그 변종, SSD, YOLO 및 그 변종까지 크게 세 종류다.

요약

- R-CNN은 R-CNN, Fast R-CNN, Faster R-CNN 이렇게 세 가지 변종이 있다.
R- CNN과 Fast R-CNN은 RoI 제안에 선택적 탐지 알고리즘을 사용하지만
Faster R- CNN은 RoI 제안을 영역 제안 신경망으로 대체해 전체 처리 과정을
딥러닝으로 구현했다.
- YOLO는 YOLOv1 이후로 현재까지 꾸준히 성능이 개선된 버전이 출시되고 있다.
- R-CNN은 경계 박스의 물체 존재 확신도와 물체 클래스 분류를 두 단계에 걸쳐
수행하는 다단계 탐지기다.
- SSD와 YOLO는 물체 존재 확신도 예측 및 물체 클래스 분류를 한 번에 마치는 단일
단계 탐지기다.
- 일반적으로 단일 단계 탐지기는 다단계 탐지기에 비해 정확도가 떨어지지만 속도가 훨씬
빠른 경향을 보인다.